

Overview of Assemblage: the microservice architecture definition process

Chris Richardson

Founder of Eventuate.io

Founder of the original CloudFoundry.com

Author of POJOs in Action and Microservices Patterns

🐦 @crichardson

chris@chrisrichardson.net

adopt.microservices.io

Agenda

- ✦ Introduction to Assemblage
- ✦ Step 1: discover system operations
- ✦ Step 2: design subdomains
- ✦ Step 3: define the service architecture
- ✦ Step 4: evaluate the architecture
- ✦ Step 5: refactor the architecture

From requirements to Microservice architecture

Functional requirements

As a consumer
I want to place an Order
So that I can

As a Restaurant
I want to accept an Order
So that I can

System quality attributes

- SLA: Reliability/Latency
- Scalability
- ...

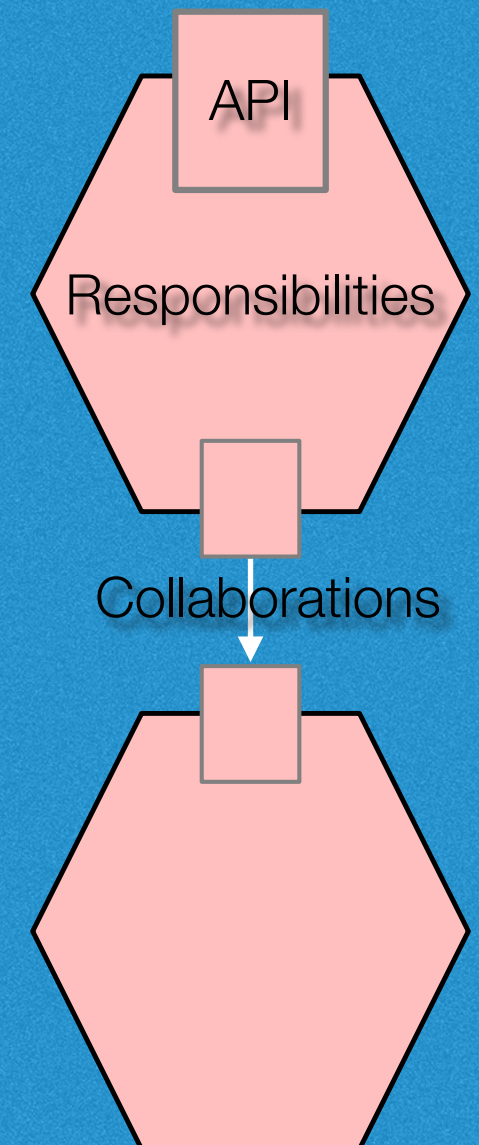
Wireframe/UI mockups



Event storming



Microservice architecture



Two levels of architecture

Service architecture

- Service definitions
 - APIs
 - Responsibilities
 - Collaborations
- Interaction styles: synchronous vs. asynchronous

Critical

vs.

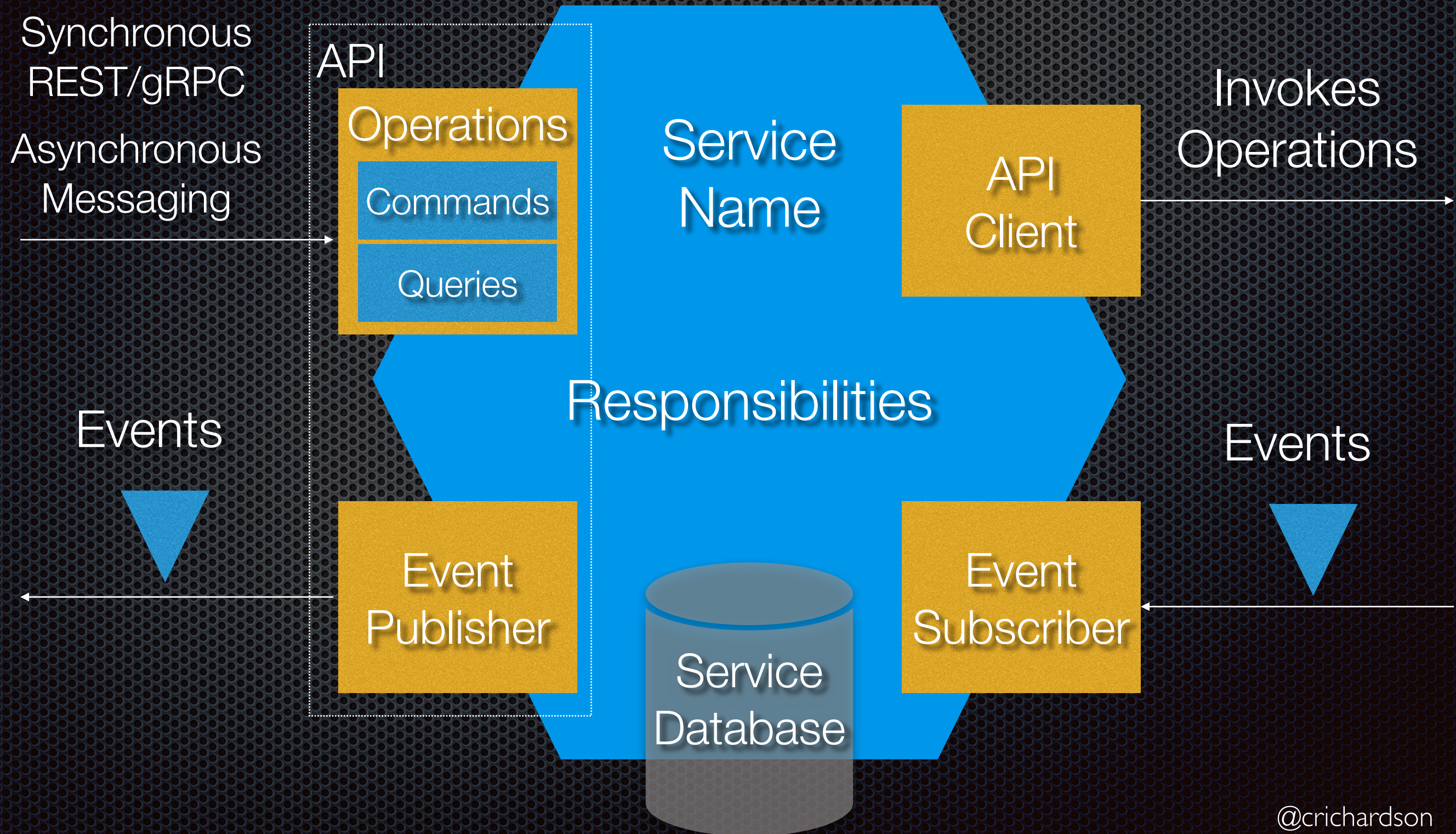
Technical architecture

- Choice of specific technologies
- Synchronous: REST vs. gRPC
- Pick message broker(s)
- Deployment technologies
- ...

Important

Focus

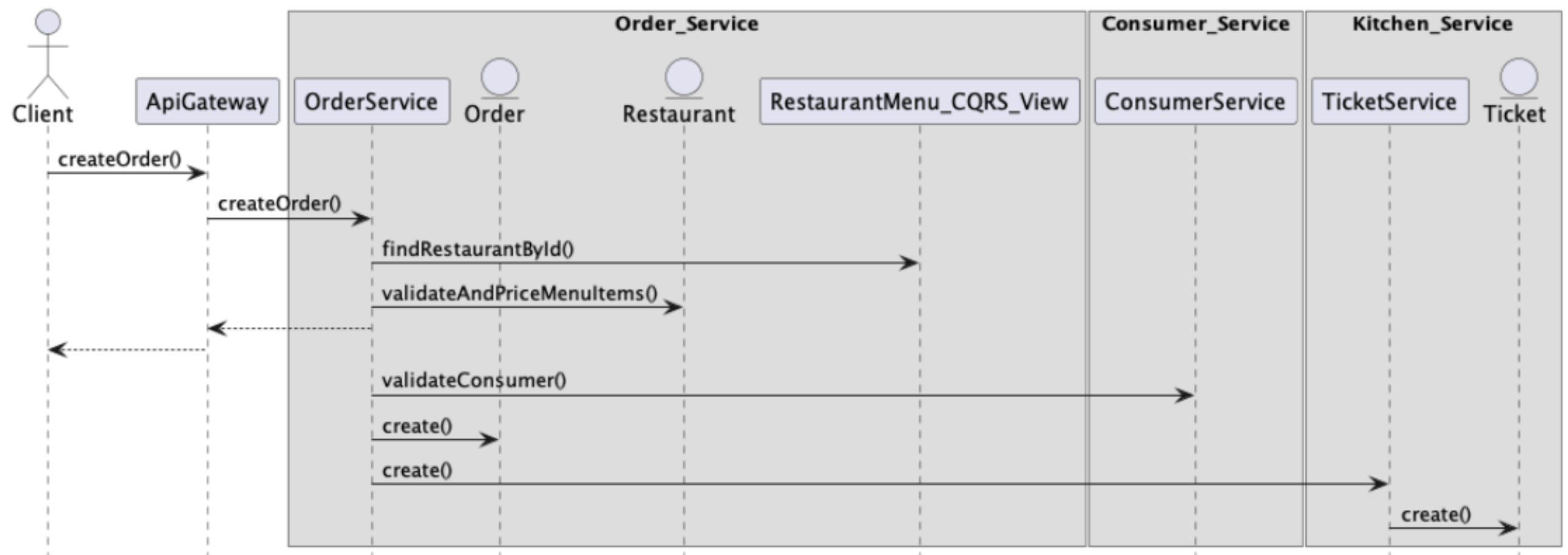
Define services: Name, APIs, responsibilities



Service design criteria

- ✦ Small:
 - ✦ Testability
 - ✦ Deployability
 - ✦ Evolvability
 - ✦ Observability
 - ✦ Loosely design-time coupled to enable Team autonomy
- AND**

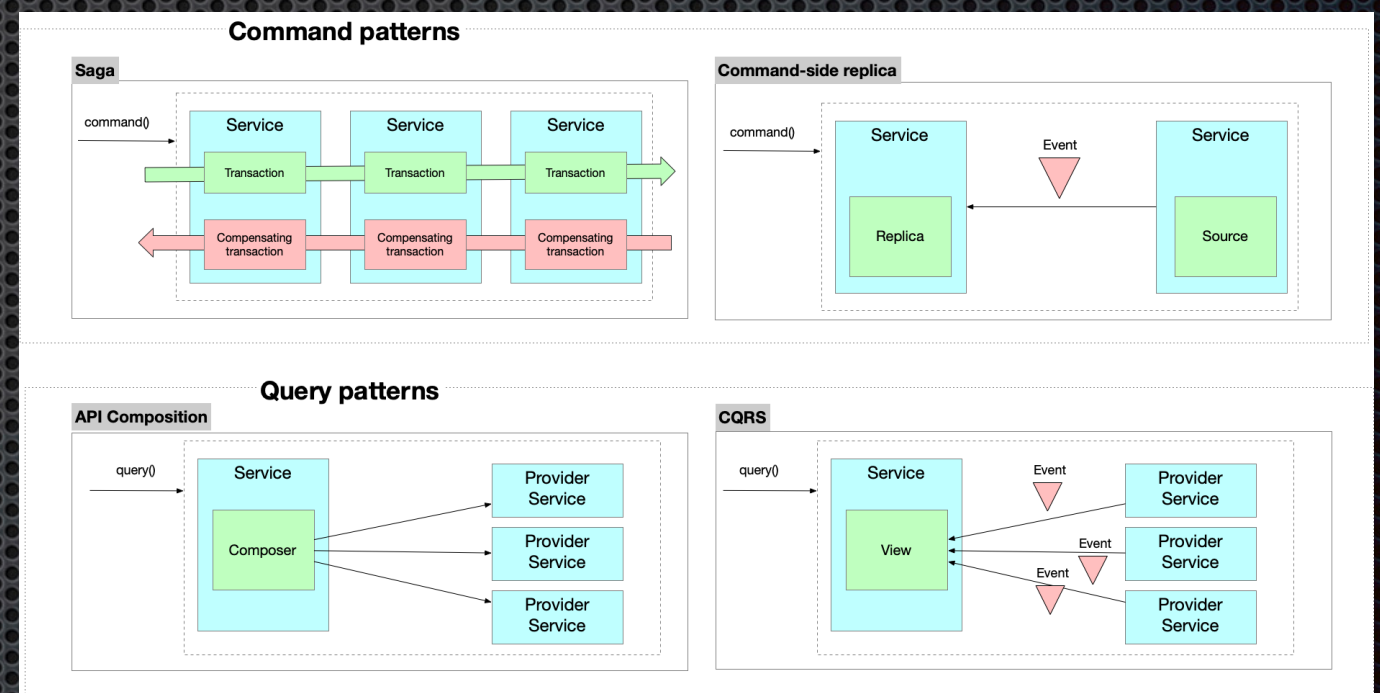
Design operations (requests): service collaboration



Collaboration patterns: Saga, Command-side replica, API
Composition, CQRS

Distributed operation design criteria

- ✦ Simplicity - maintainability (ease of understanding and development)
- ✦ Efficiency - performance
- ✦ Runtime coupling - availability, latency
- ✦ Prefer ACID over BASE - maintainability

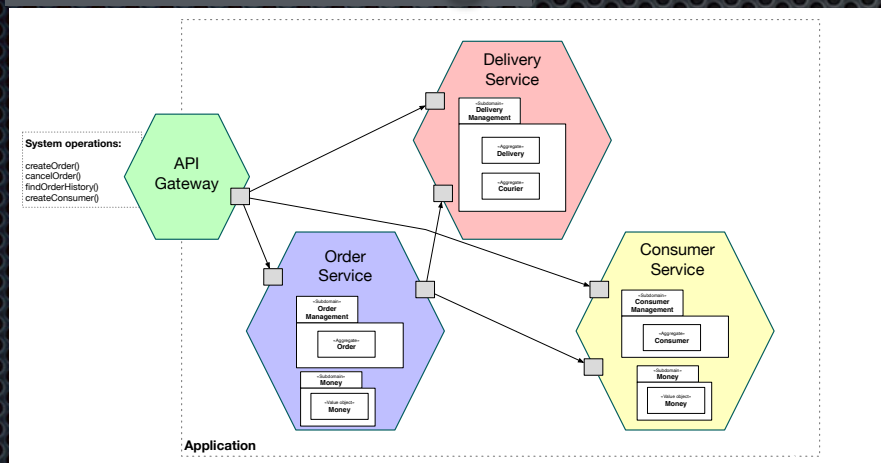


<https://microservices.io/post/patterns/2023/07/29/service-collaboration-patterns.html>

<https://microservices.io/post/architecture/2023/03/26/dark-energy-dark-matter-force-descriptions.html>

Designing an architecture = making trade-offs

Service design



Constrains

- Simplicity
- Efficiency
- Runtime coupling
- Prefer ACID over BASE

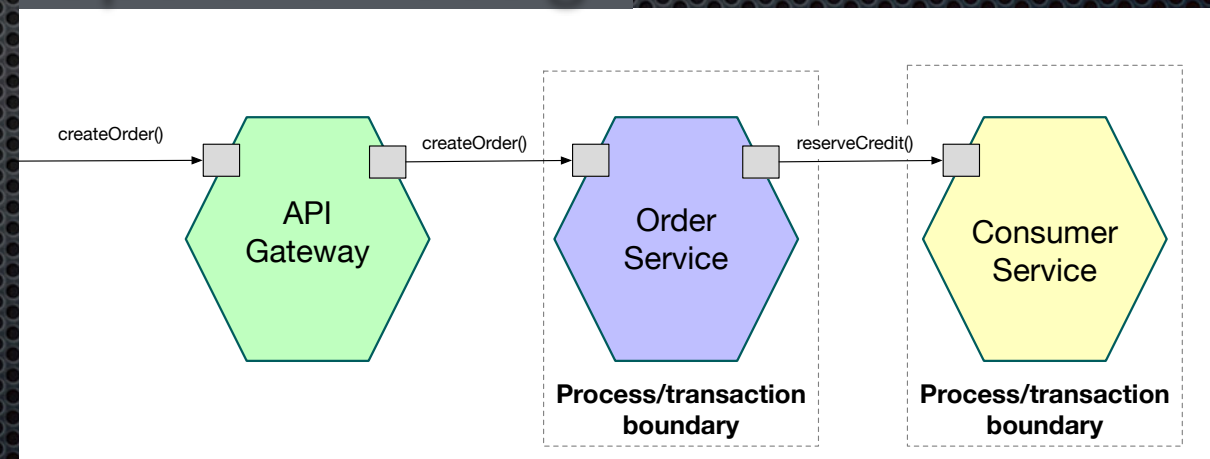
Conflict

Determines

Determines

Determines

Operation design



- Loose-coupling
- Testability
- Deployability

Requirements for fast flow

The starting point

0

Functional requirements

As a consumer
I want to place an Order
So that I can

As a Restaurant
I want to accept an Order
So that I can

System quality attributes

- SLA: Reliability/Latency
- Scalability
- ...

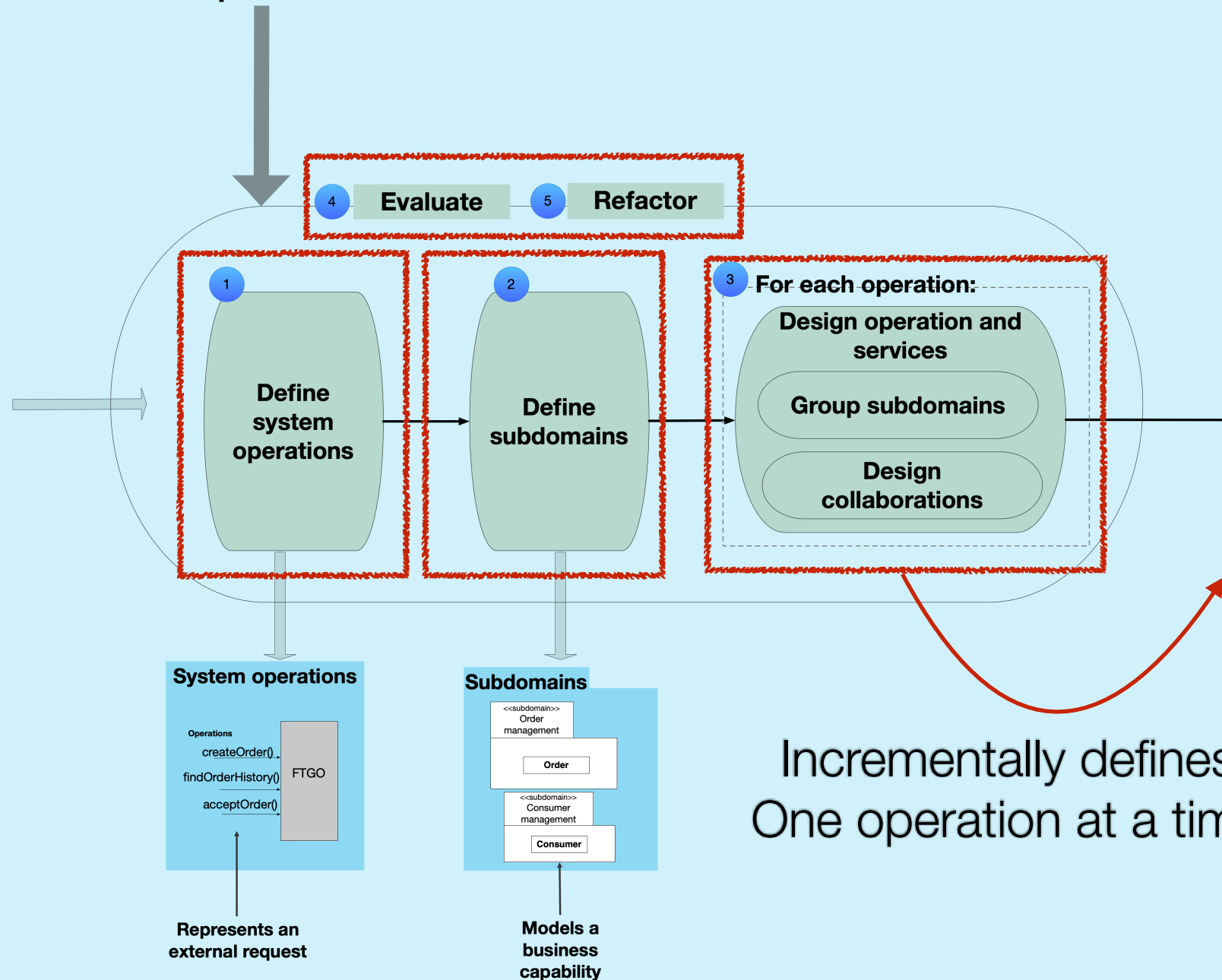
Wireframe/UI mockups

Delivery Address → Available Restaurants → Restaurant Menu

Event storming

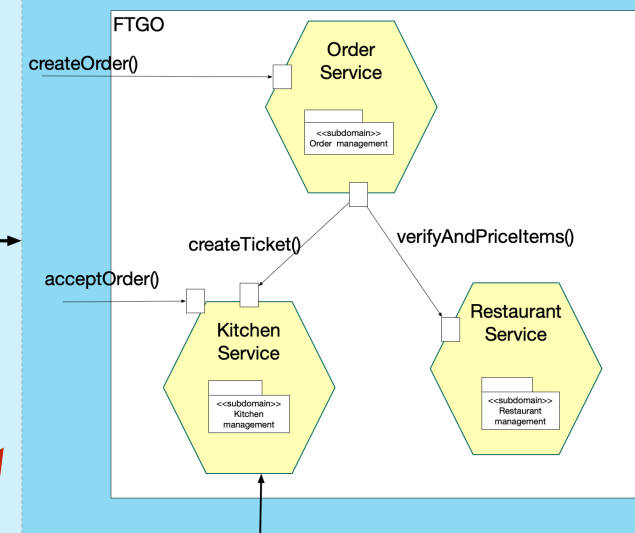


The process



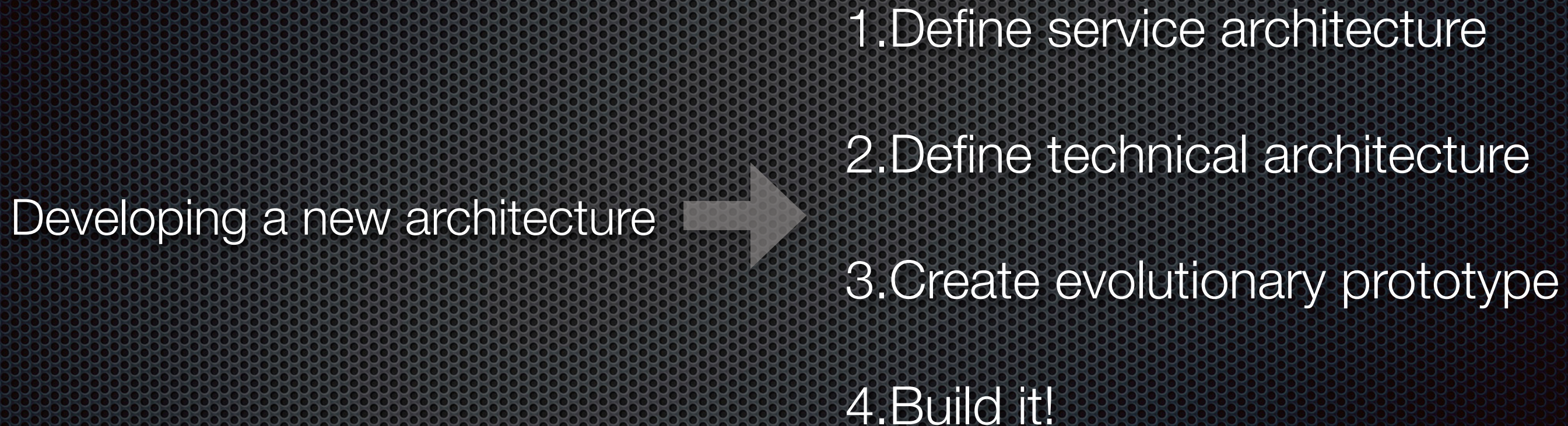
The result

Service architecture



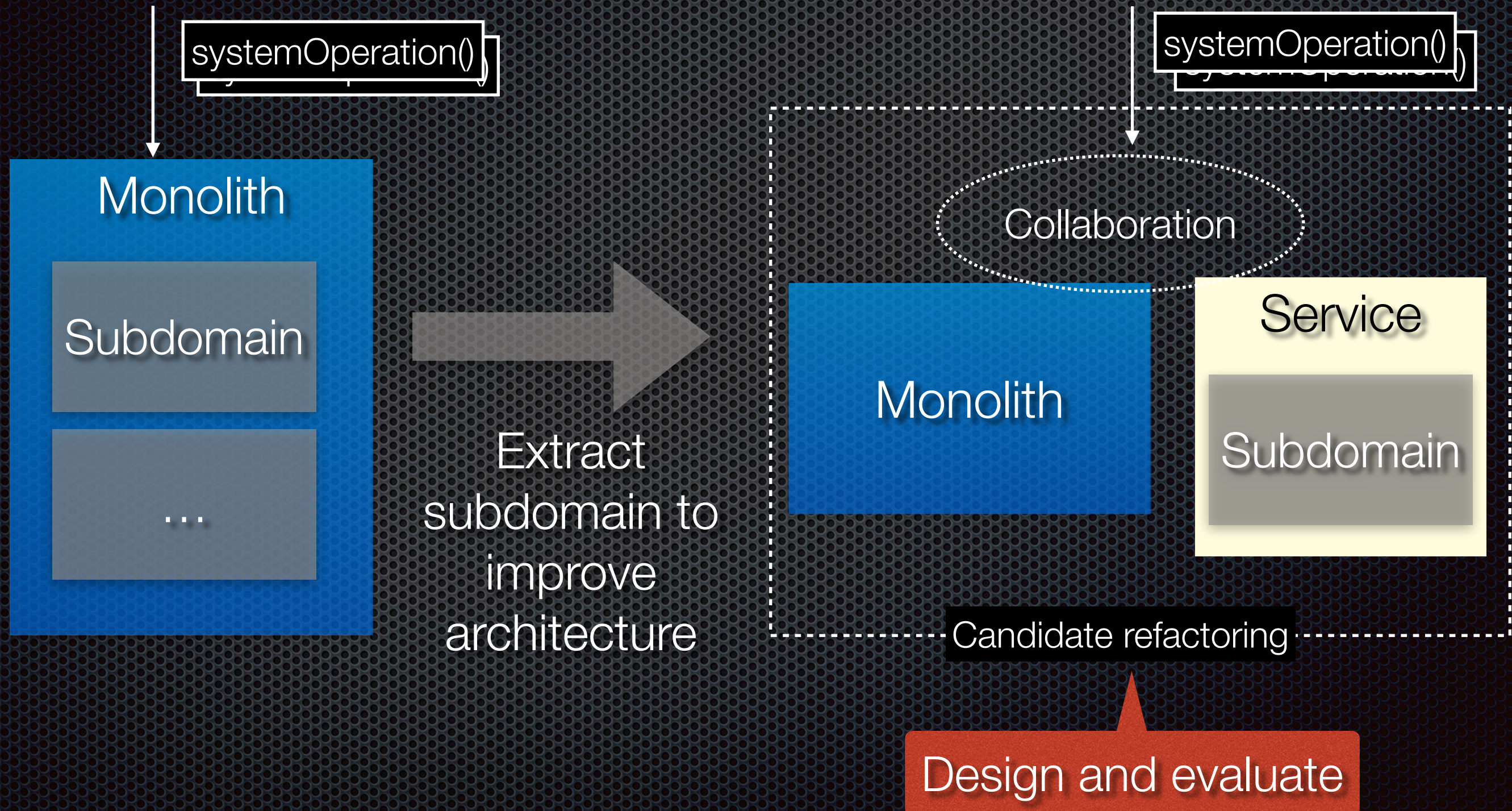
Service = group of subdomains

When to use Assemblage: Greenfield development*



* or brainstorming a target architecture when refactoring a monolith

When to use Assemblage: extracting services from a monolith



When to use Assemblage: Evolving an architecture

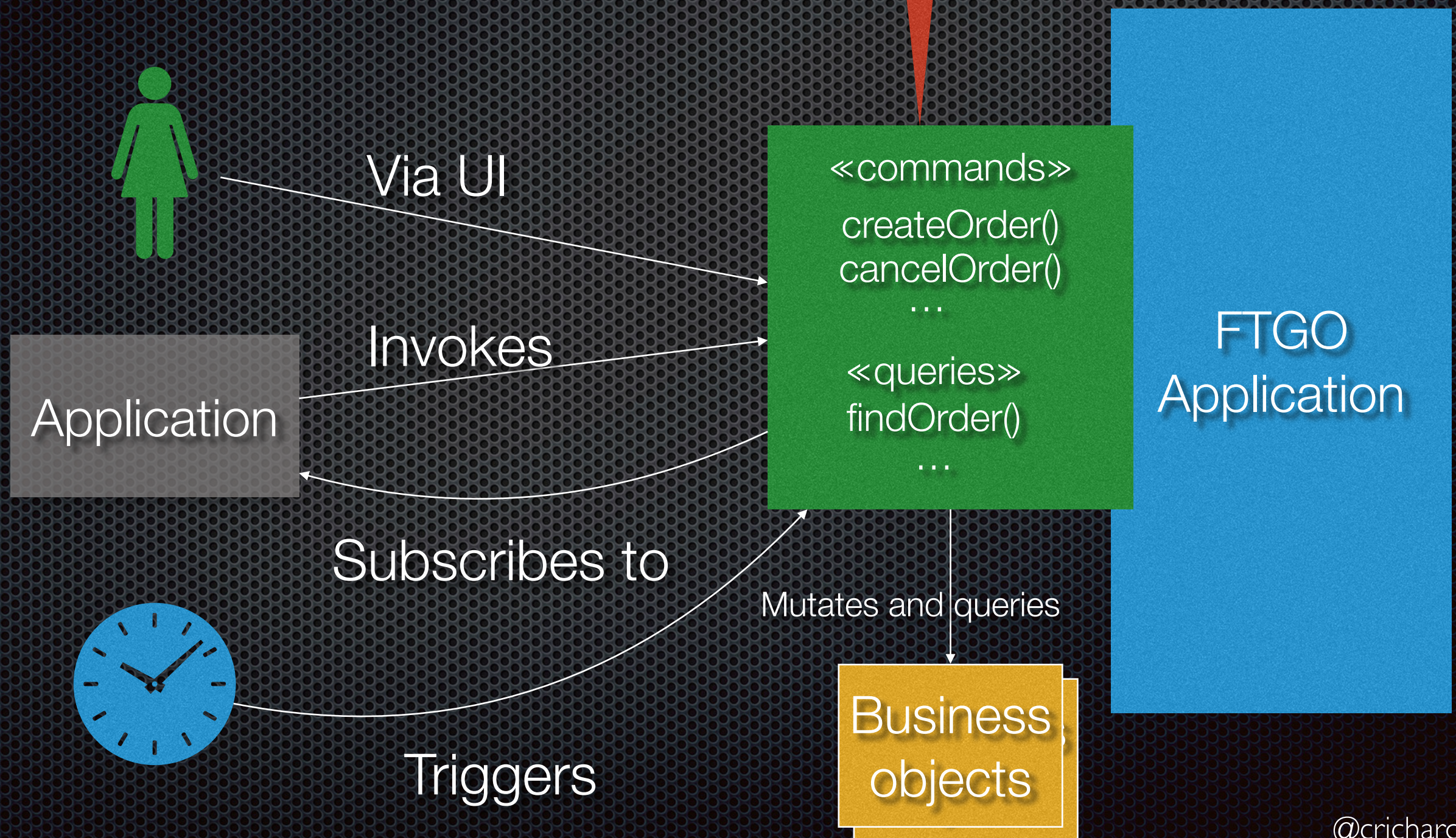
- Design new/changed features
- Constantly evaluate and refactor

Agenda

- ✦ Introduction to Assemblage
- ✦ Step 1: discover system operations
- ✦ Step 2: design subdomains
- ✦ Step 3: define the service architecture
- ✦ Step 4: evaluate the architecture
- ✦ Step 5: refactor the architecture

Step #1: defining system operations

Model application behavior



Examples of system operations

- `createCustomer()` - creates customer
- `createRestaurant()` - creates restaurant
- `createCourier()` - creates courier that delivers orders
- `createOrder()` - creates an order
- `acceptTicket()` - confirms pick up time for order and schedules delivery

Example implementation technologies

- ✦ REST endpoint, e.g. HTTP POST /orders
- ✦ gRPC service method
- ✦ External events, e.g.
 - ✦ Twilio webhooks - SMS message received, ...
 - ✦ Stripe webhooks- payment succeeded, ...
- ✦ Scheduler: Quartz, Kubernetes Cronjob, ...

Agenda

- ✦ Introduction to Assemblage
- ✦ Step 1: discover system operations
- ✦ Step 2: design subdomains
- ✦ Step 3: define the service architecture
- ✦ Step 4: evaluate the architecture
- ✦ Step 5: refactor the architecture

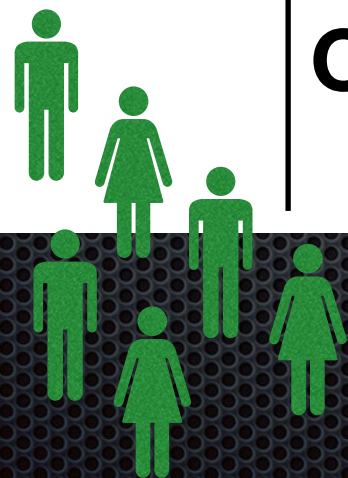
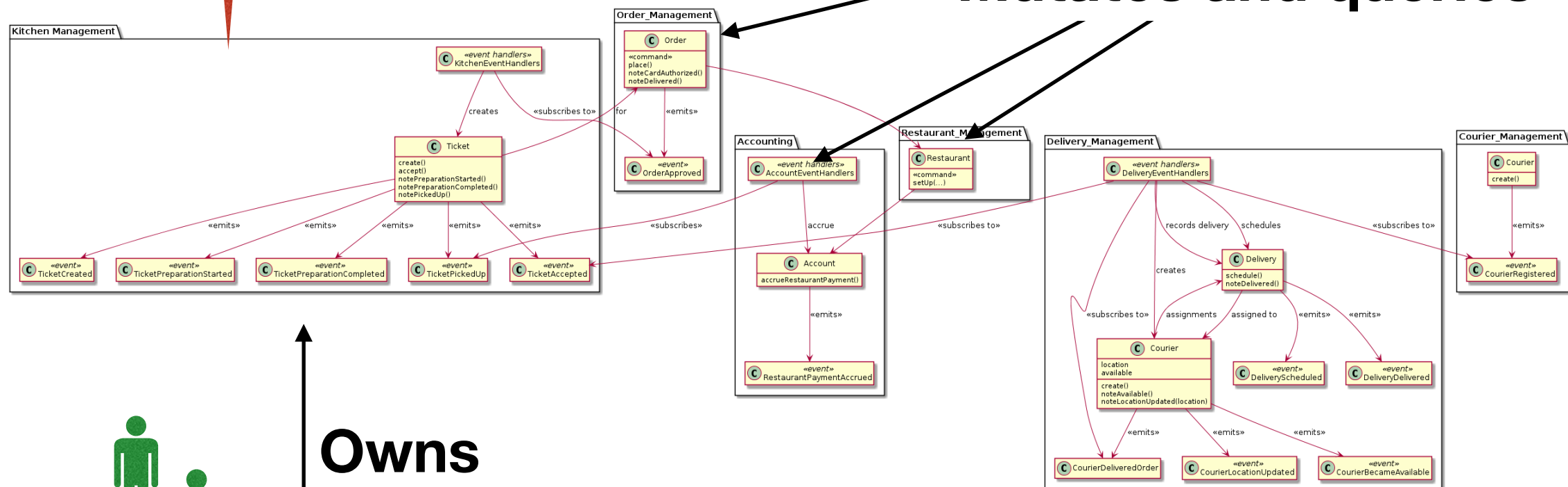
Step #2: define subdomains

Models/implements business capability

createOrder()

FTGO Application

Mutates and queries



Kitchen Team

Subdomains: Eg. Java classes and packages that implement business logic

Customer Management

```
main
├── java
│   └── io.eventuate.examples.tram.ordersandcustomers.customers
│       ├── domain
│       │   ├── Customer
│       │   ├── CustomerCreditLimitExceededException
│       │   └── CustomerRepository
│       ├── service
│       └── web
│           ├── CustomerConfiguration
│           └── CustomerServiceMain
```

```
public class CustomerService {

    private Logger logger = LoggerFactory.getLogger(getClass());

    private CustomerRepository customerRepository;

    private DomainEventPublisher domainEventPublisher;

    @Entity
    @Table(name="Customer")
    @Access(AccessType.FIELD)
    public class Customer {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;
        private String name;

        @Embedded
        private Money creditLimit;

        // ...
    }
}
```

Order Management

```
order-service
├── build
├── out
├── src
│   └── main
│       ├── java
│       │   └── io.eventuate.examples.tram.ordersandcustomers
│       │       ├── domain
│       │       │   ├── Order
│       │       │   ├── OrderRepository
│       │       │   └── PendingOrderCantBeCancelledException
│       │       ├── service
│       │       └── web
│       │           ├── OrderConfiguration
│       │           └── OrderServiceMain
```

```
public class OrderService {

    private final DomainEventPublisher domainEventPublisher;
    private final OrderRepository orderRepository;

    @Entity
    @Table(name="orders")
    @Access(AccessType.FIELD)
    public class Order {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Long id;

        @Enumerated(EnumType.STRING)
        private OrderState state;

        @Embedded
        private OrderDetails orderDetails;

        // ...
    }
}
```


DDD is helpful but

- The goal is to define a service architecture - not create detailed models

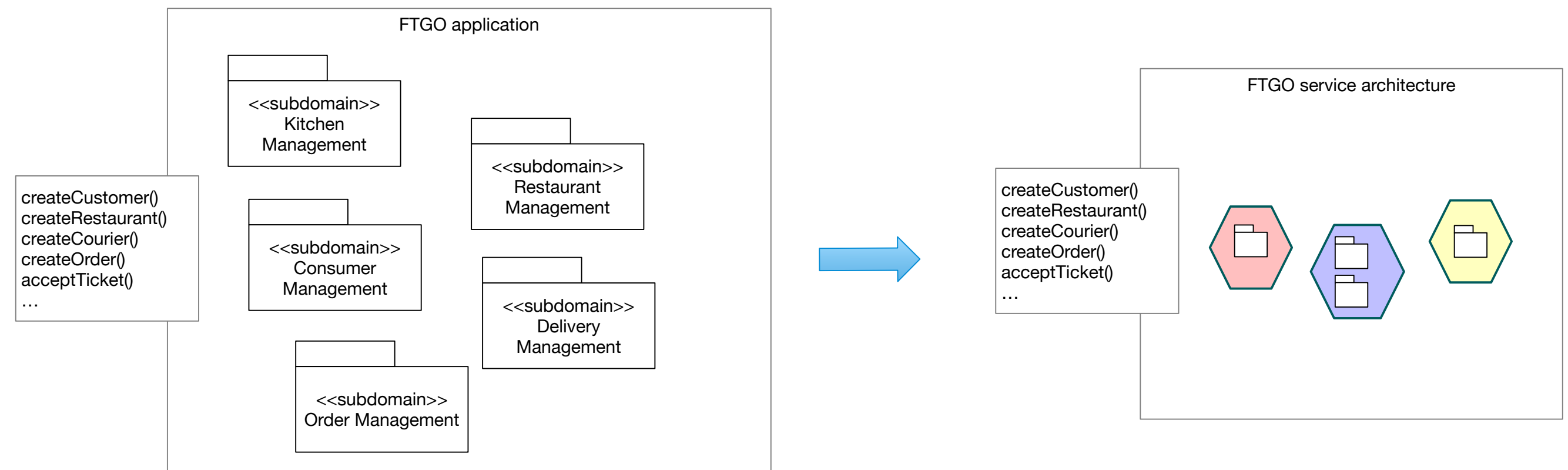
THEREFORE

- Identify a subdomain's aggregates and determine operation $\Leftrightarrow$ subdomain relationship
- Identify a subdomain's characteristics: DDD type, complexity, business criticality, development regulations, ...

Agenda

- ✦ Introduction to Assemblage
- ✦ Step 1: discover system operations
- ✦ Step 2: design subdomains
- ✦ Step 3: define the service architecture
- ✦ Step 4: evaluate the architecture
- ✦ Step 5: refactor the architecture

Step #3: Define service architecture



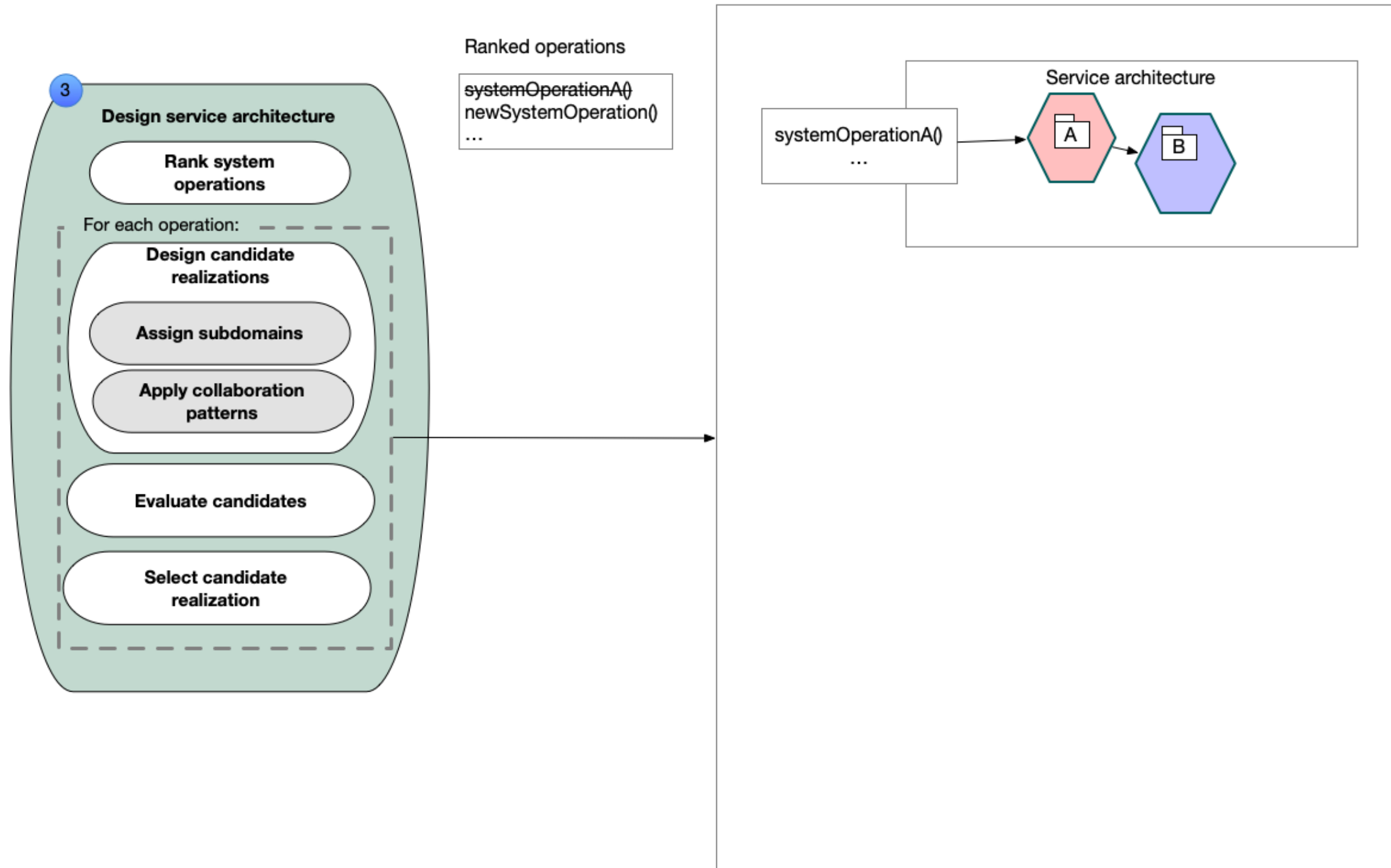
Challenge: many possible service architectures

- Need a service architecture that resolves dark energy and dark matter forces

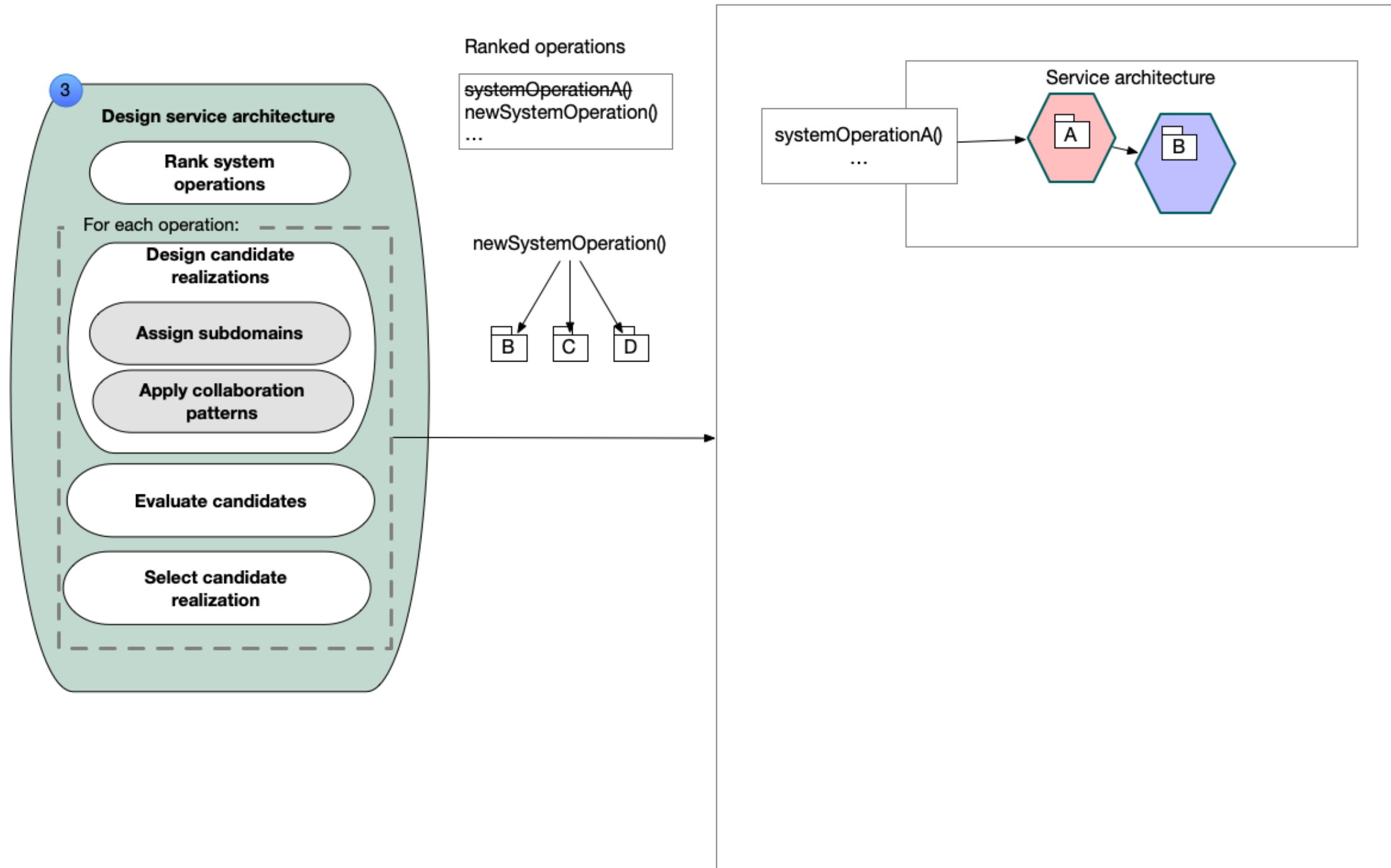
BUT

- Many ways to partition subdomains
 - e.g. 3 subdomains => 5 possible partitions
 - https://en.wikipedia.org/wiki/Bell_number
- Potentially many designs for a distributed operation

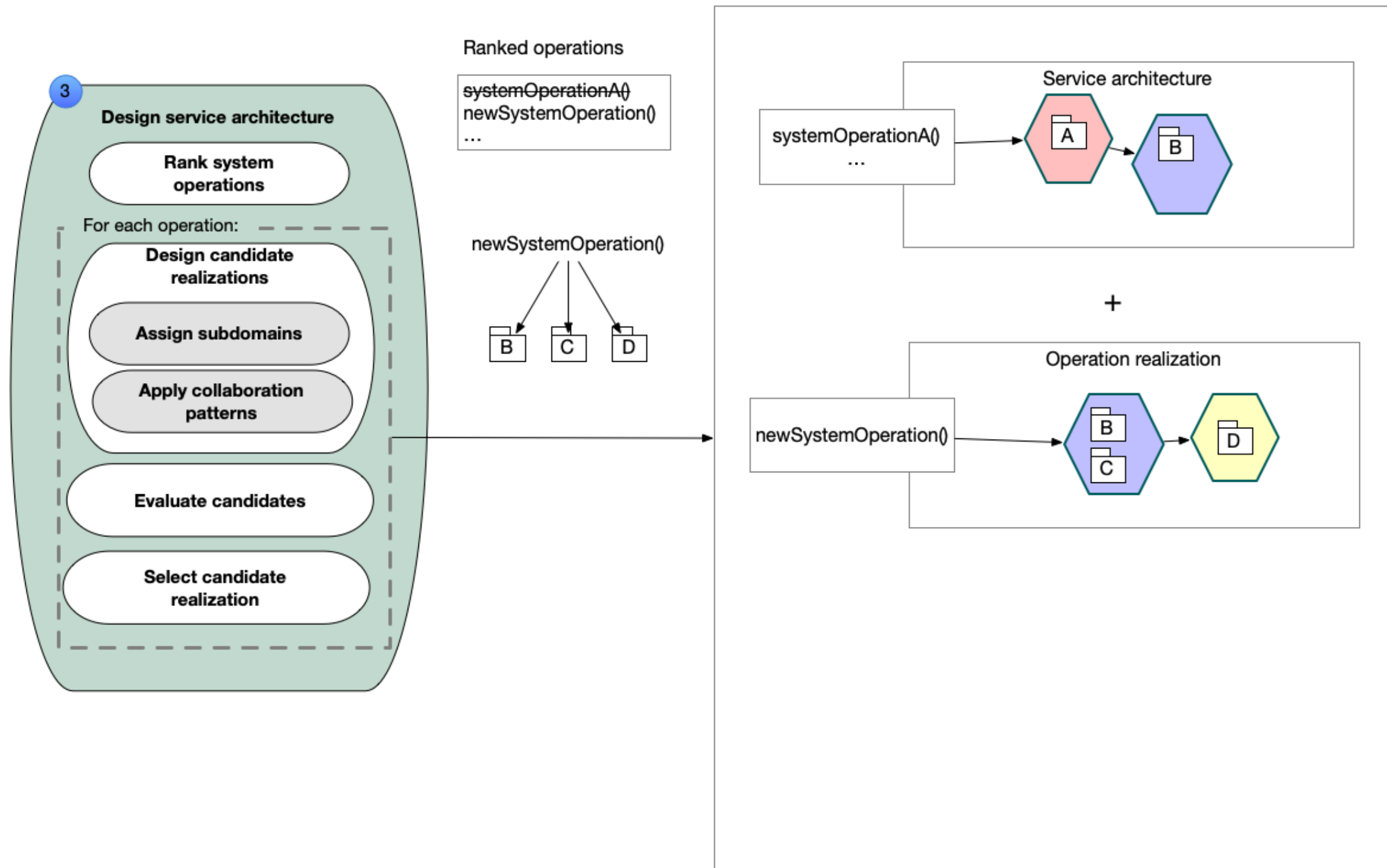
Solution: Iteratively design service architecture...



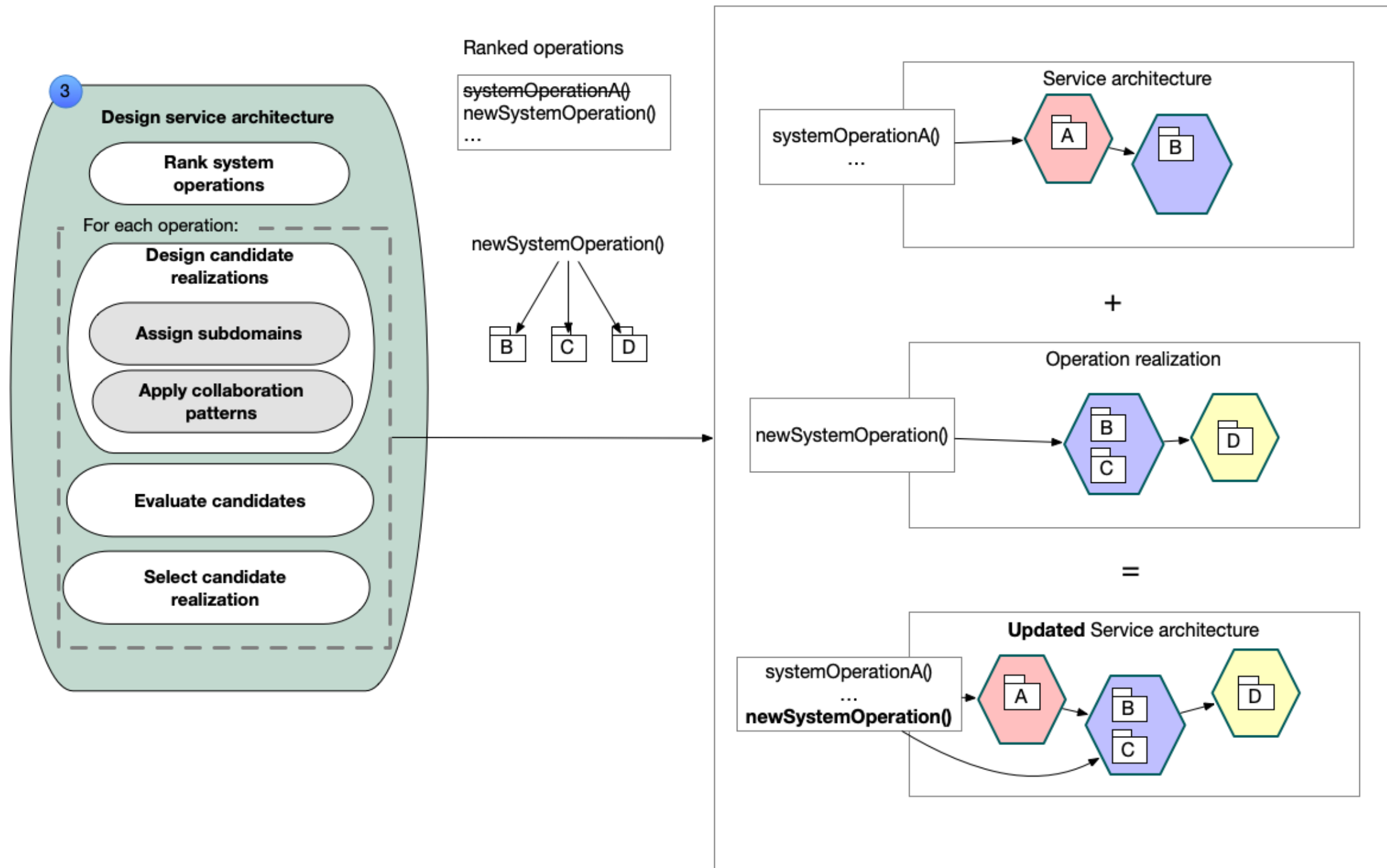
...Solution: Iteratively design service architecture...



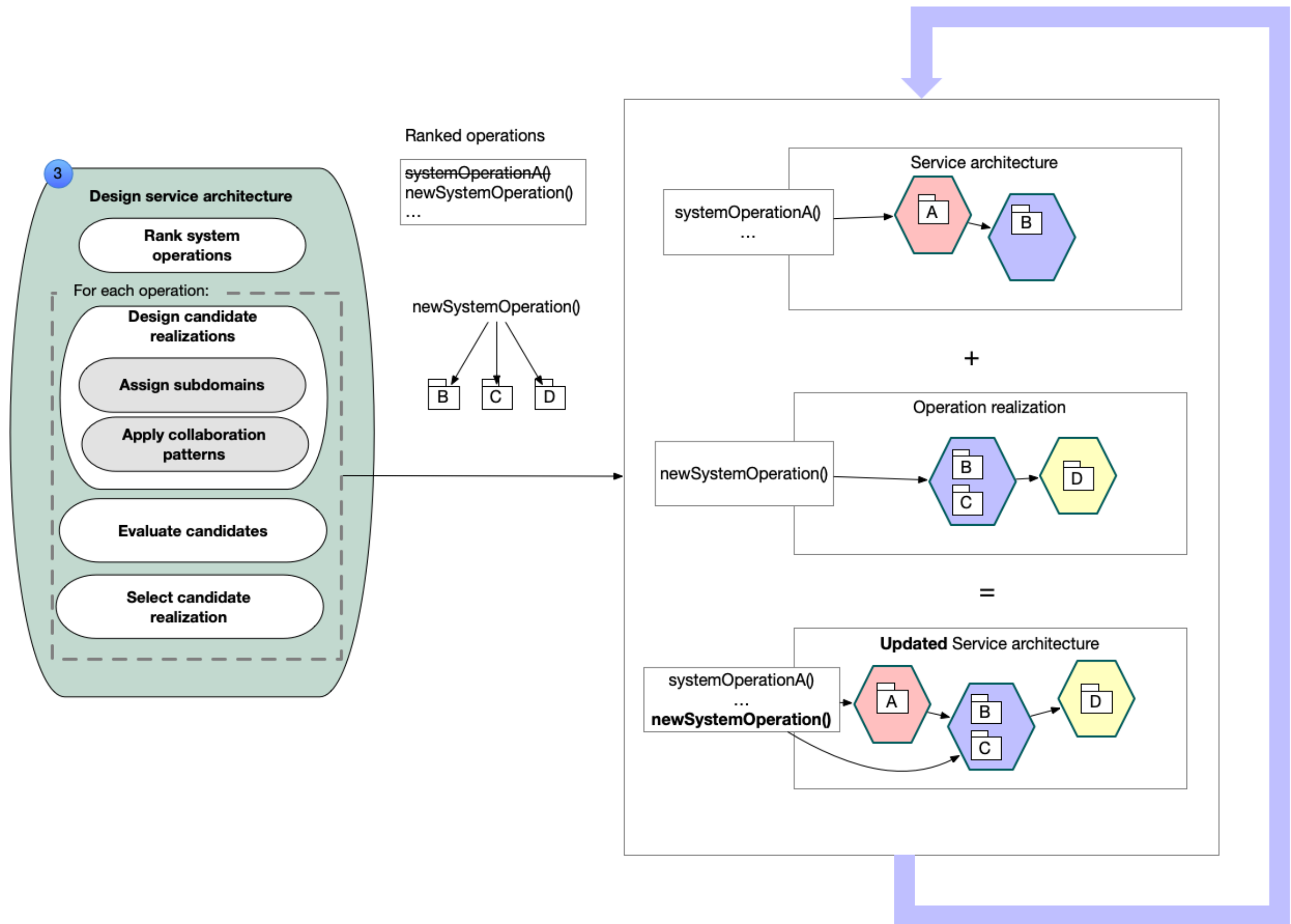
...Solution: Iteratively design service architecture...



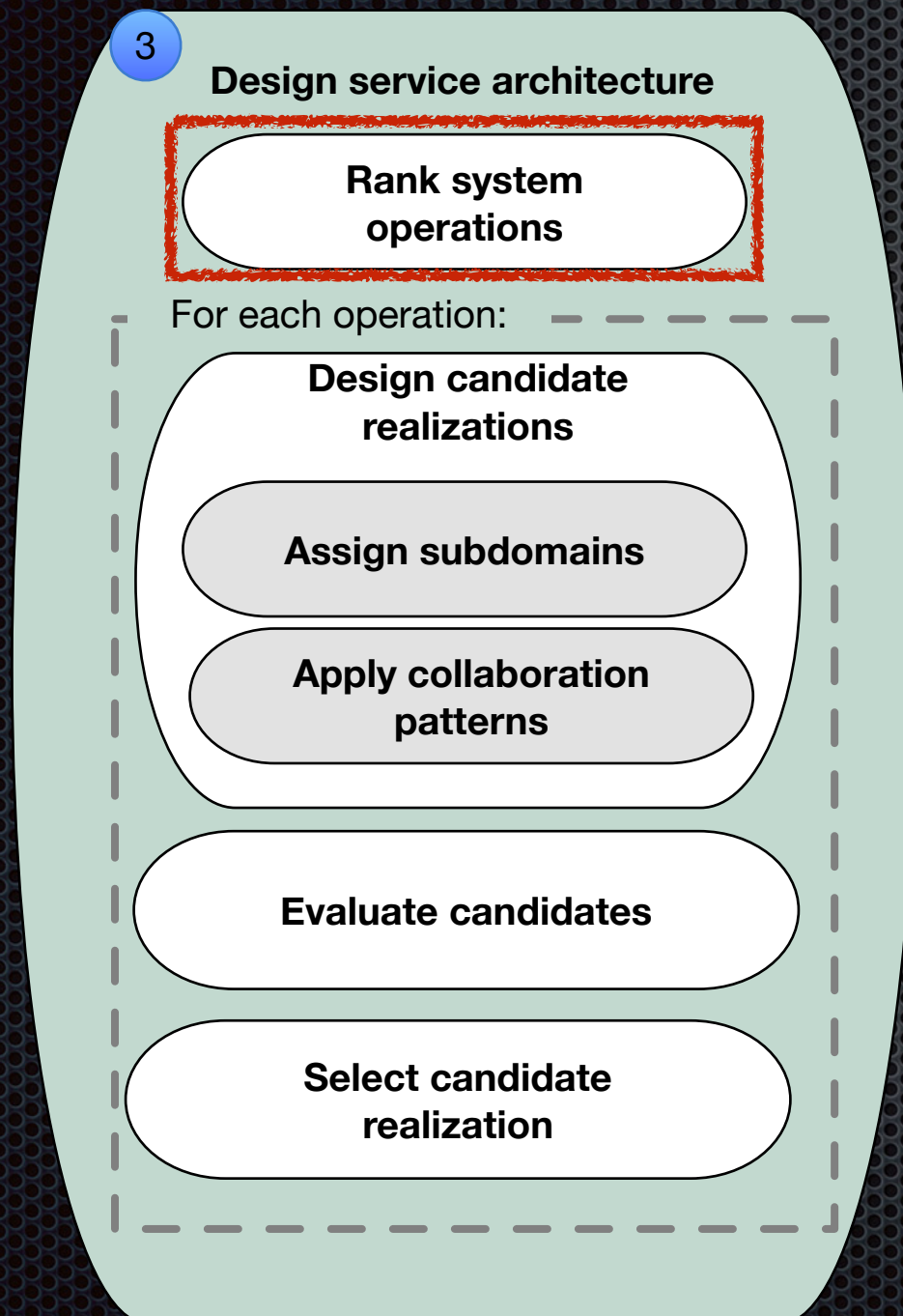
...Solution: Iteratively design service architecture...



...Solution: Iteratively design service architecture



Ranking system operations by importance



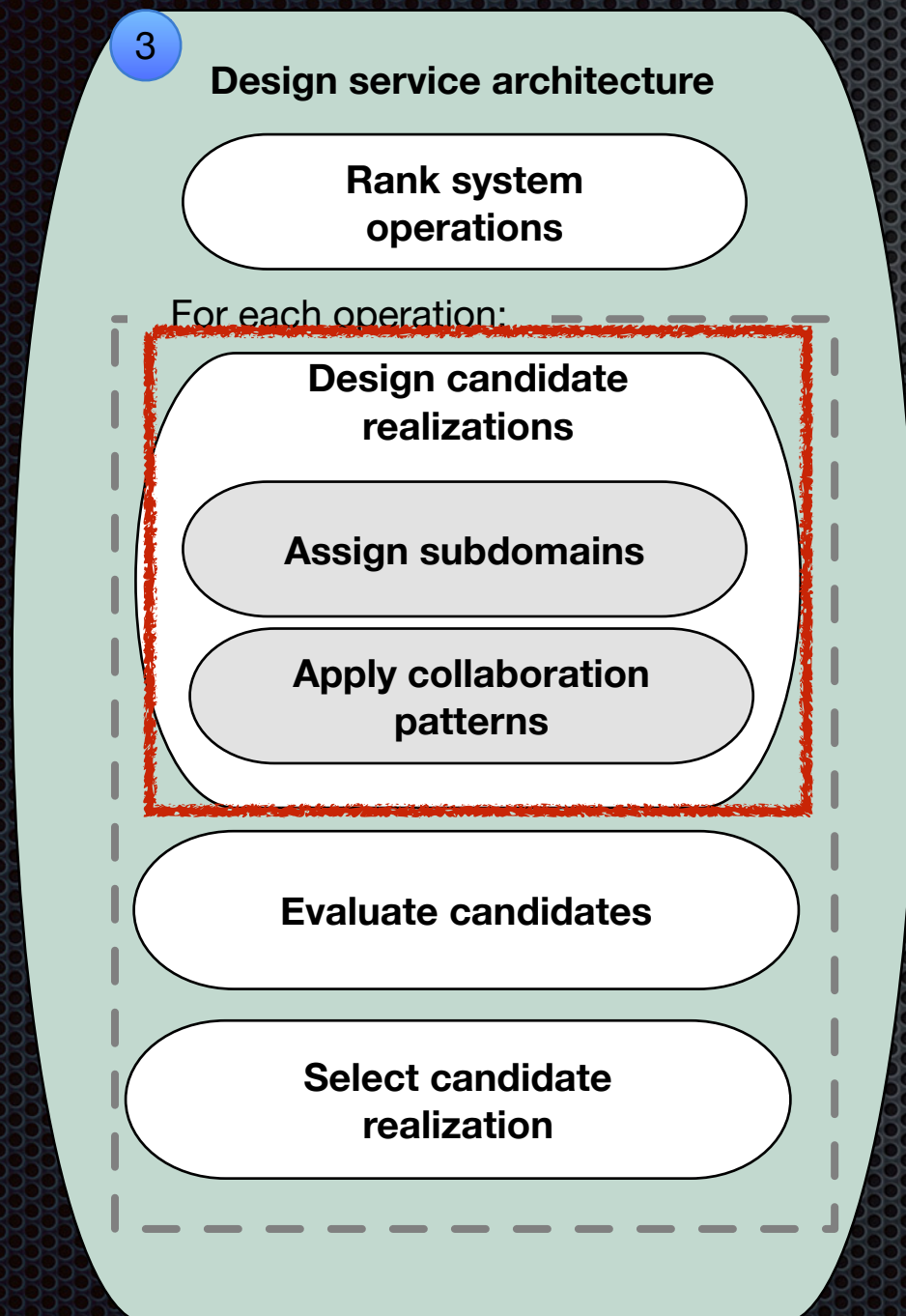
Example criteria:

- ✦ Business criticality
- ✦ Technical risk:
 - ✦ Complexity
 - ✦ Large data volumes
- ✦ ...

FTGO example ranking

1. `createOrder()` - creates an order
2. `acceptTicket()` - confirms pick up time for order and schedules delivery
3. `createCustomer()` - creates customer
4. `createRestaurant()` - creates restaurant
5. `createCourier()` - creates courier that delivers orders

About operation realizations

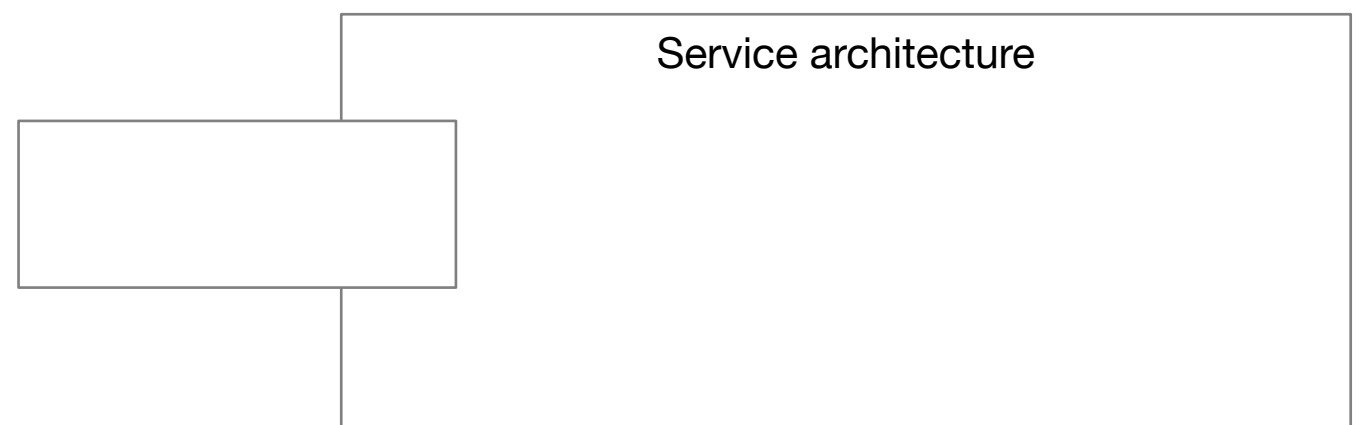


- ✦ *Service architecture fragment* that implements a system operation
- ✦ Consists of **additive** architecture modifications
 - ✦ Create (new) service
 - ✦ Assign (unassigned) subdomain to service
 - ✦ Add service elements: operations, event handlers, etc.

In the beginning: empty service architecture

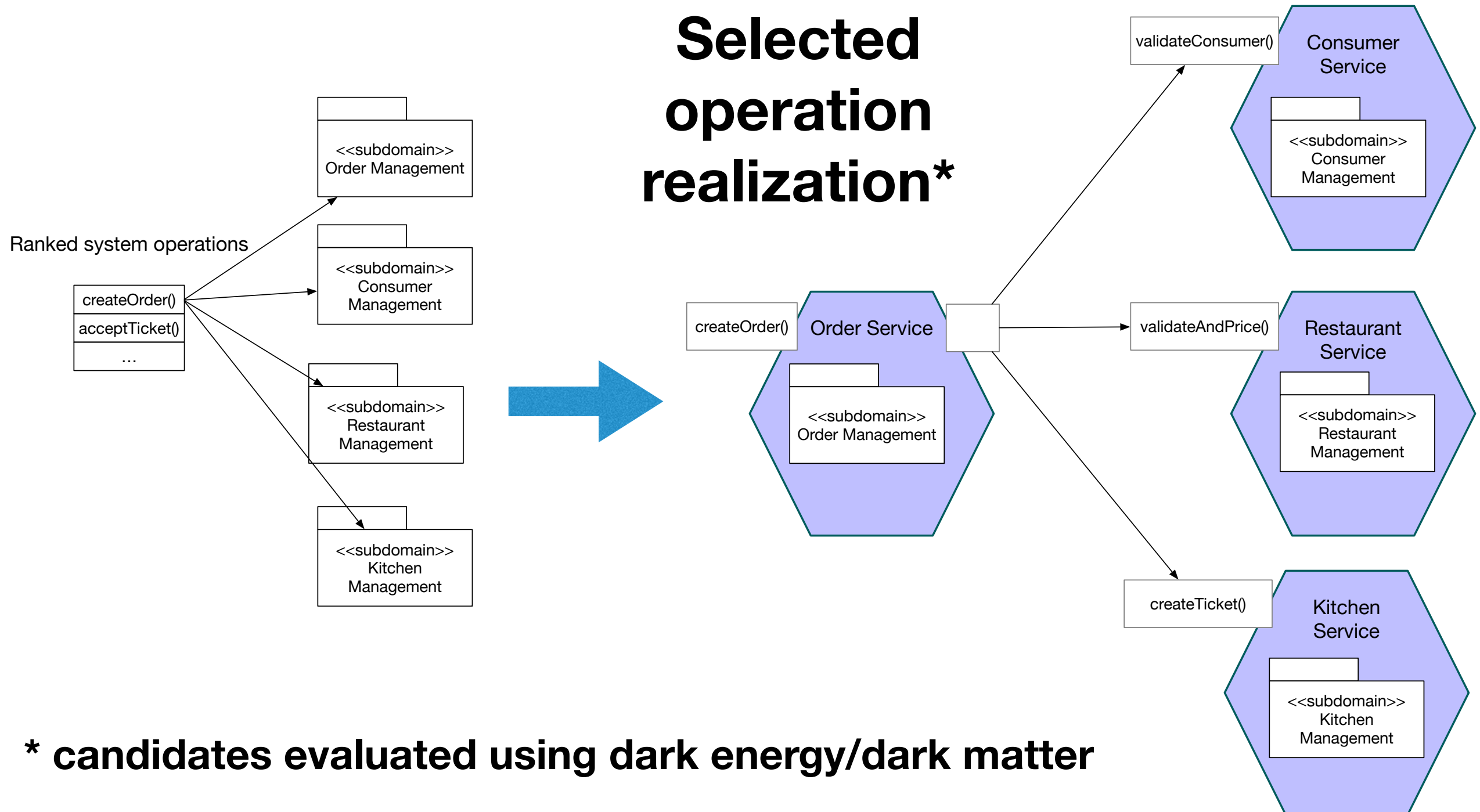
Ranked system operations

createOrder()
acceptTicket()
...

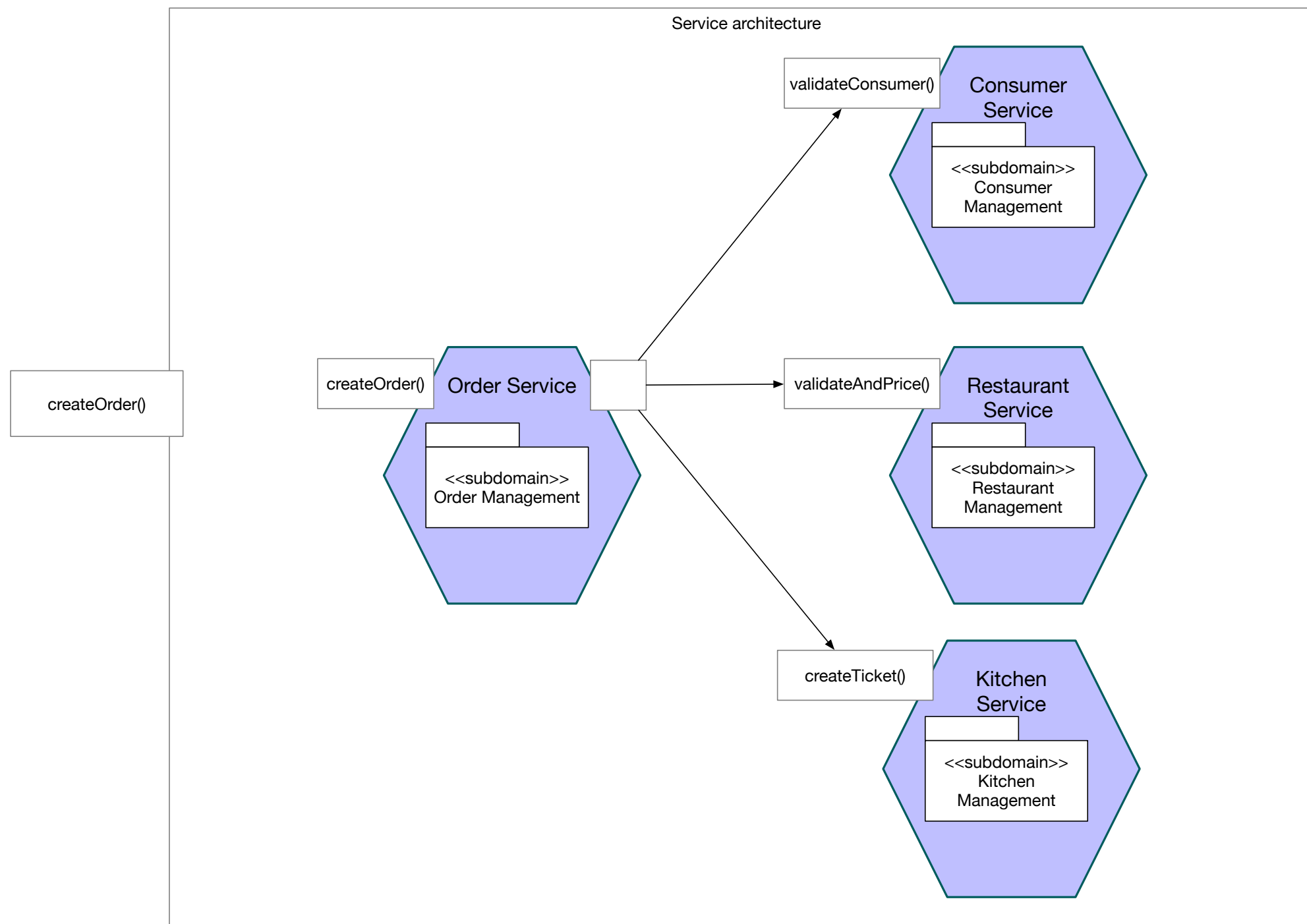


Designing createOrder() realization...

Selected operation realization*

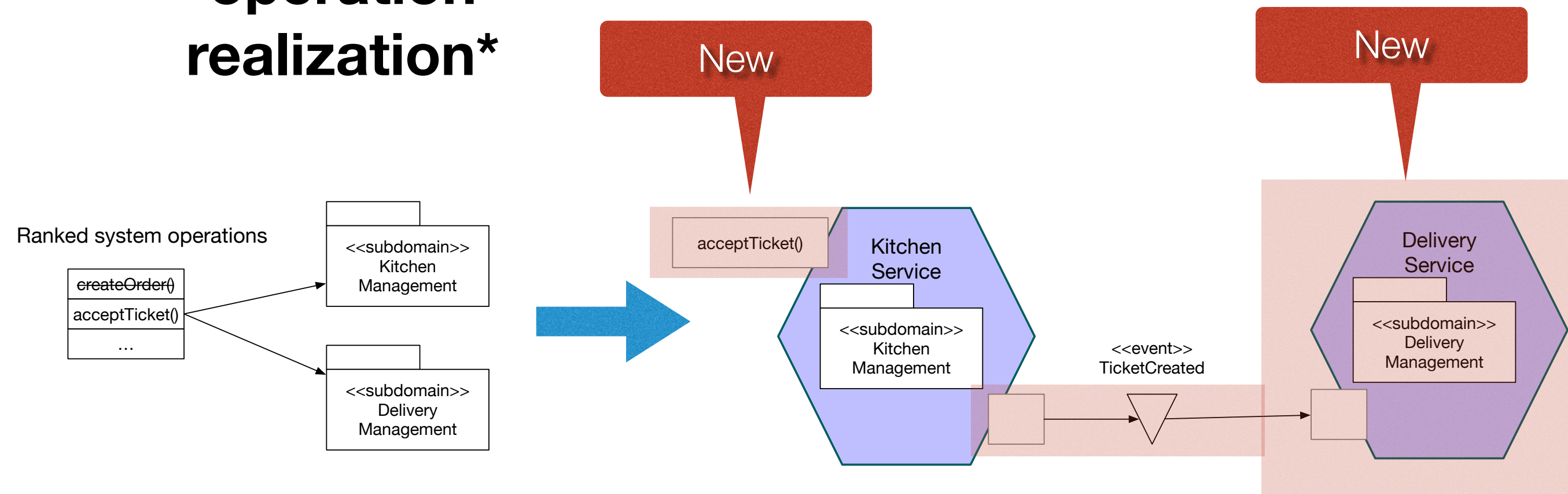


... Resulting service architecture



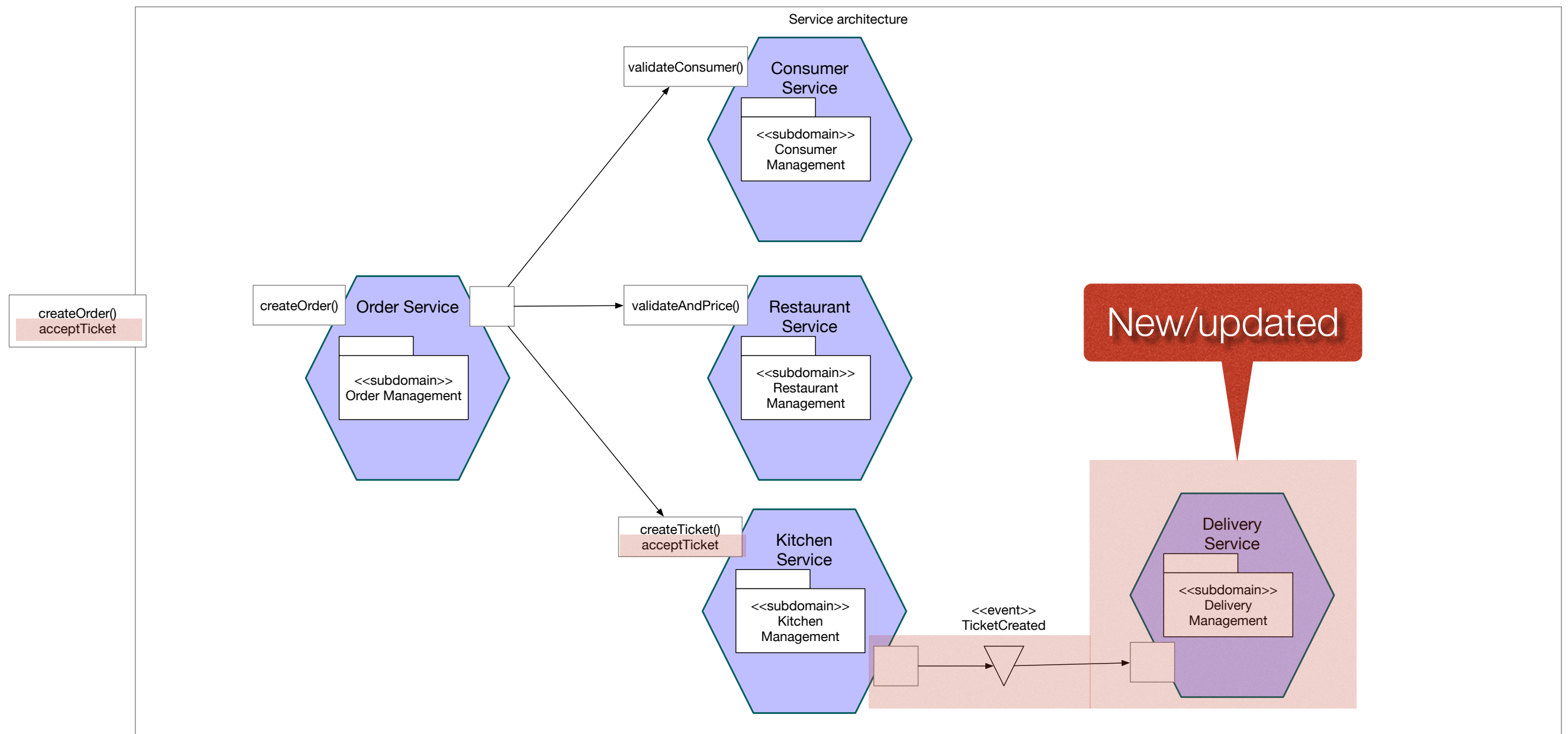
Designing acceptTicket() realization...

Selected operation realization*



*** candidates evaluated using dark energy/dark matter**

... Resulting service architecture



Designing operation realization

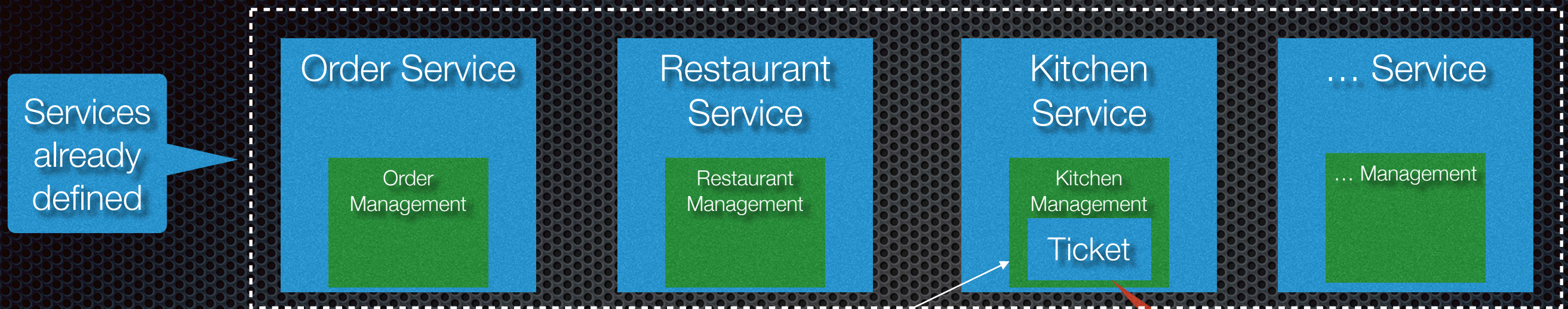
=

Generate candidate designs

Evaluate and pick the best

one

Example: designing acceptTicket()



Ranked operations

createOrder()

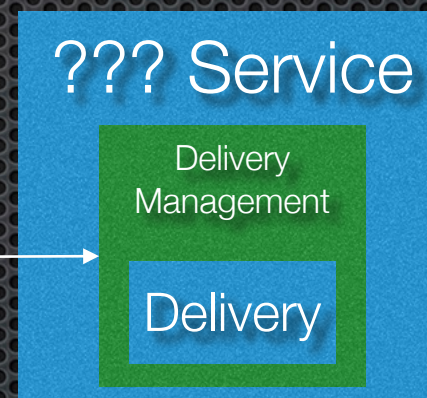
acceptTicket(ticketID)

Pre-conditions:
Exists **Ticket** t
 where t.ID = ticketID
 and t.state = PENDING
Post-conditions:
t.state became ACCEPTED
Delivery d was created
d.state became SCHEDULED
d... Became ...

noteAccepted()

Already assigned

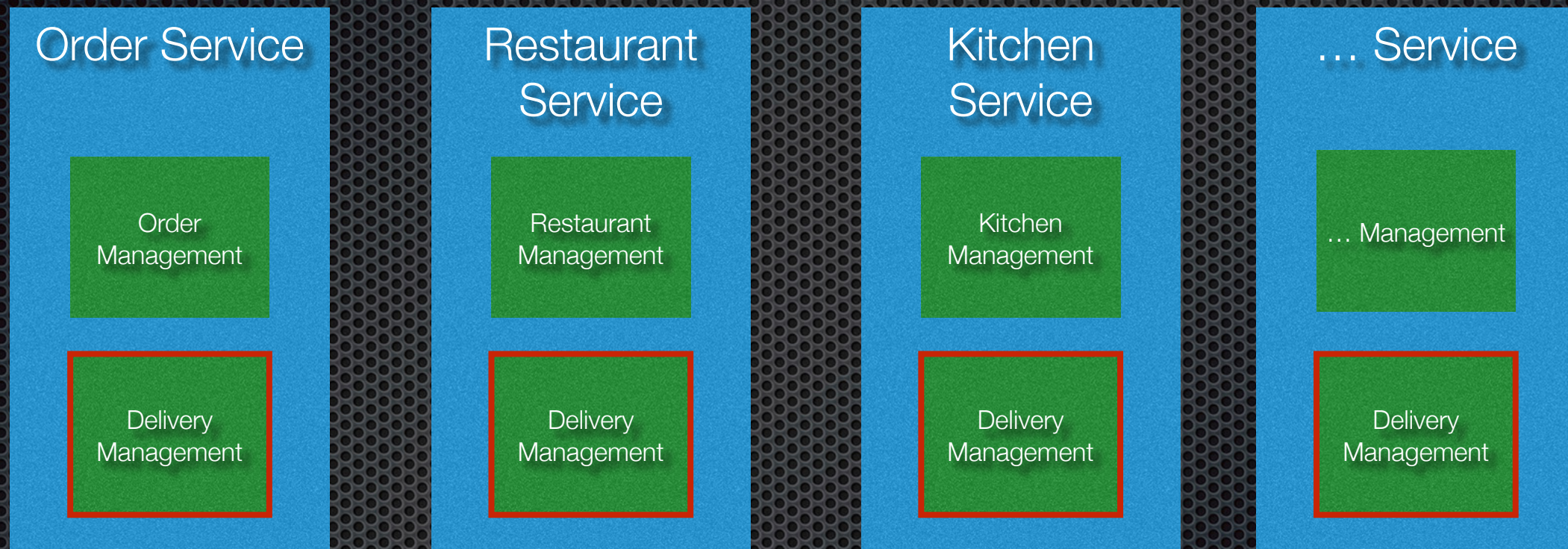
scheduleDelivery()



Must assign to either
existing service or new
service

Multiple possible assignments of Delivery Management

Existing

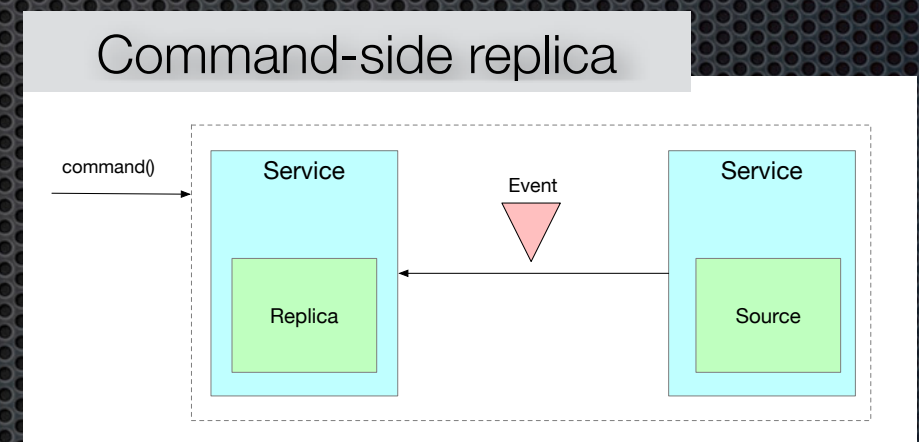
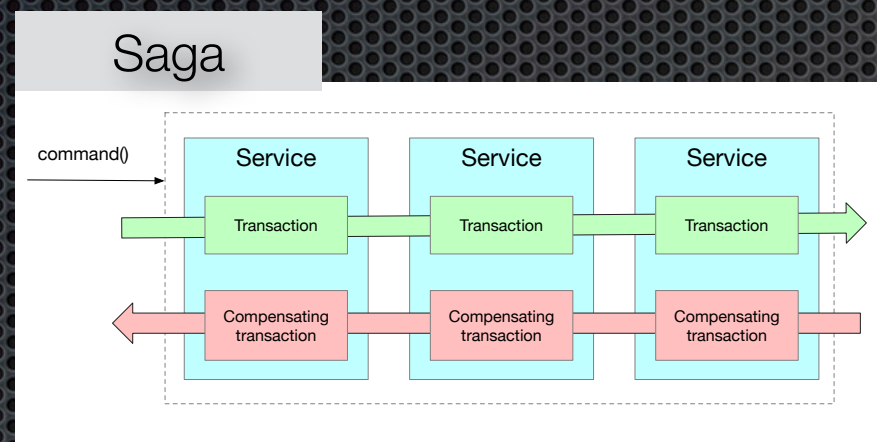


New

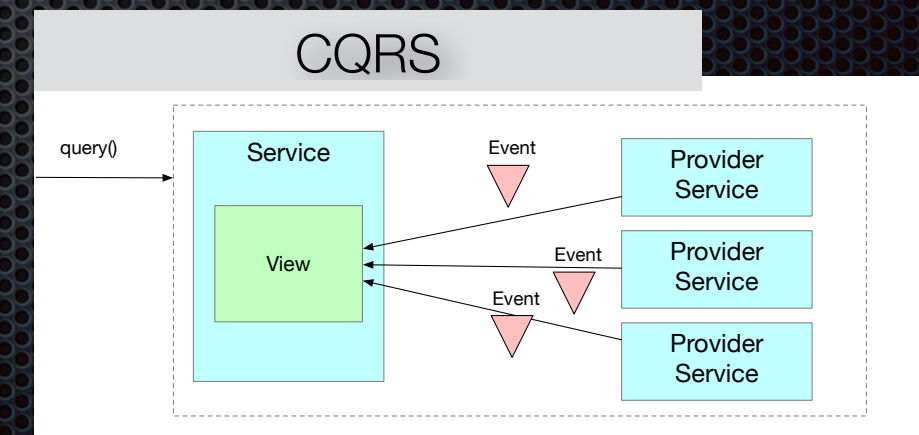
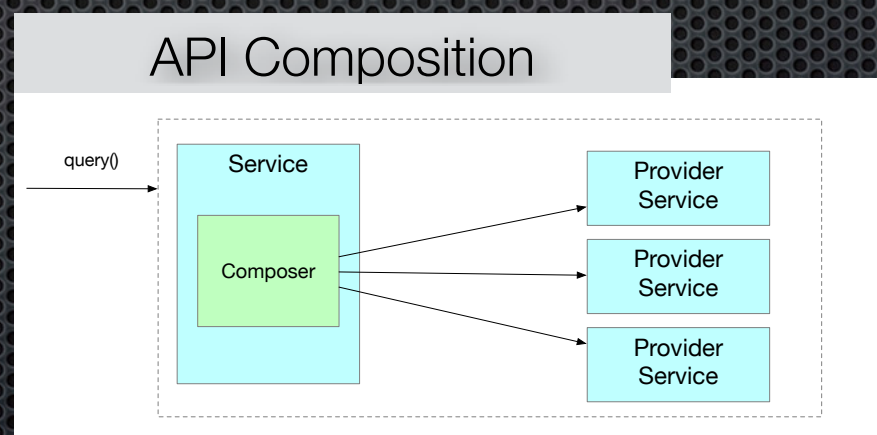


Design distributed operations using service collaboration patterns

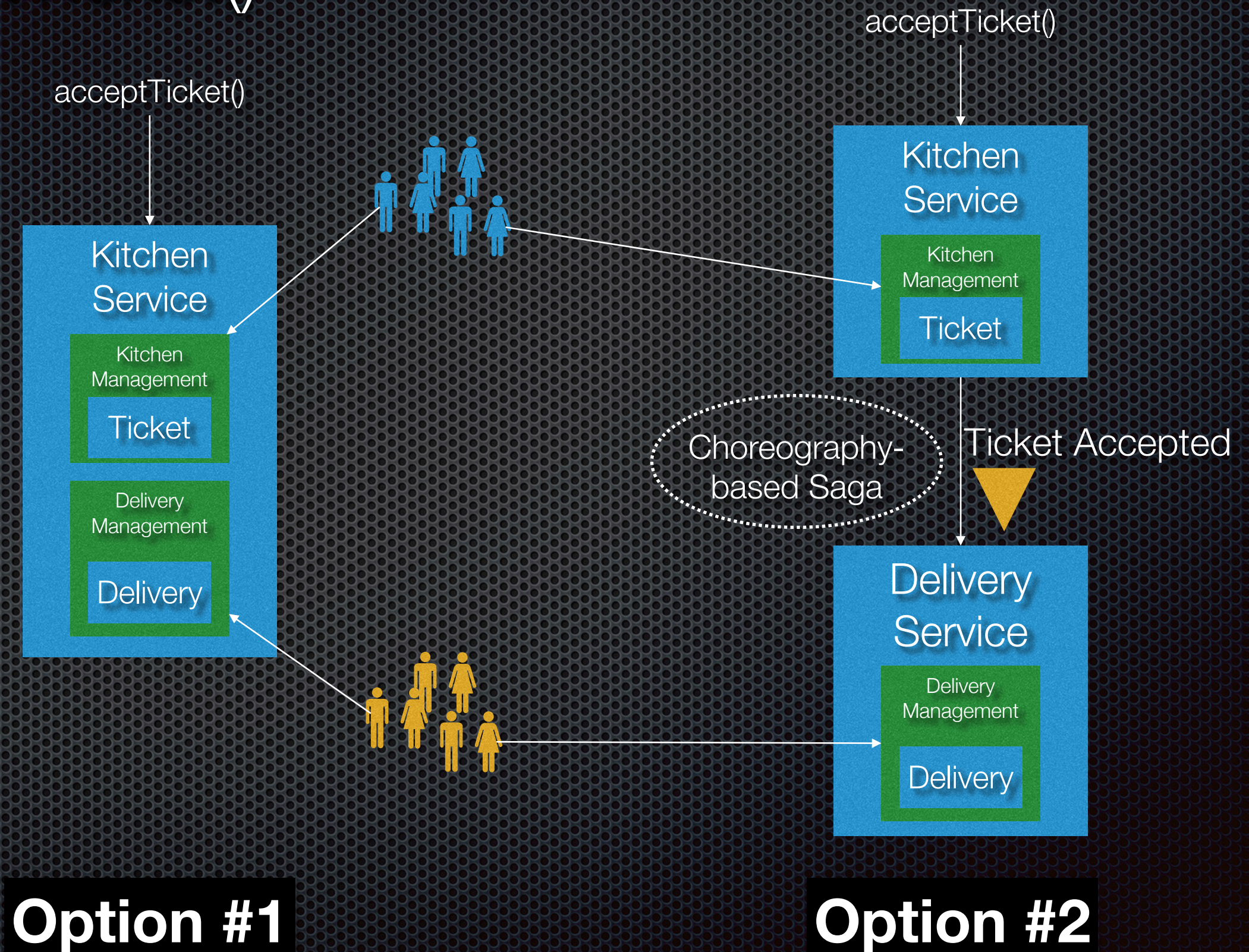
Commands



Queries

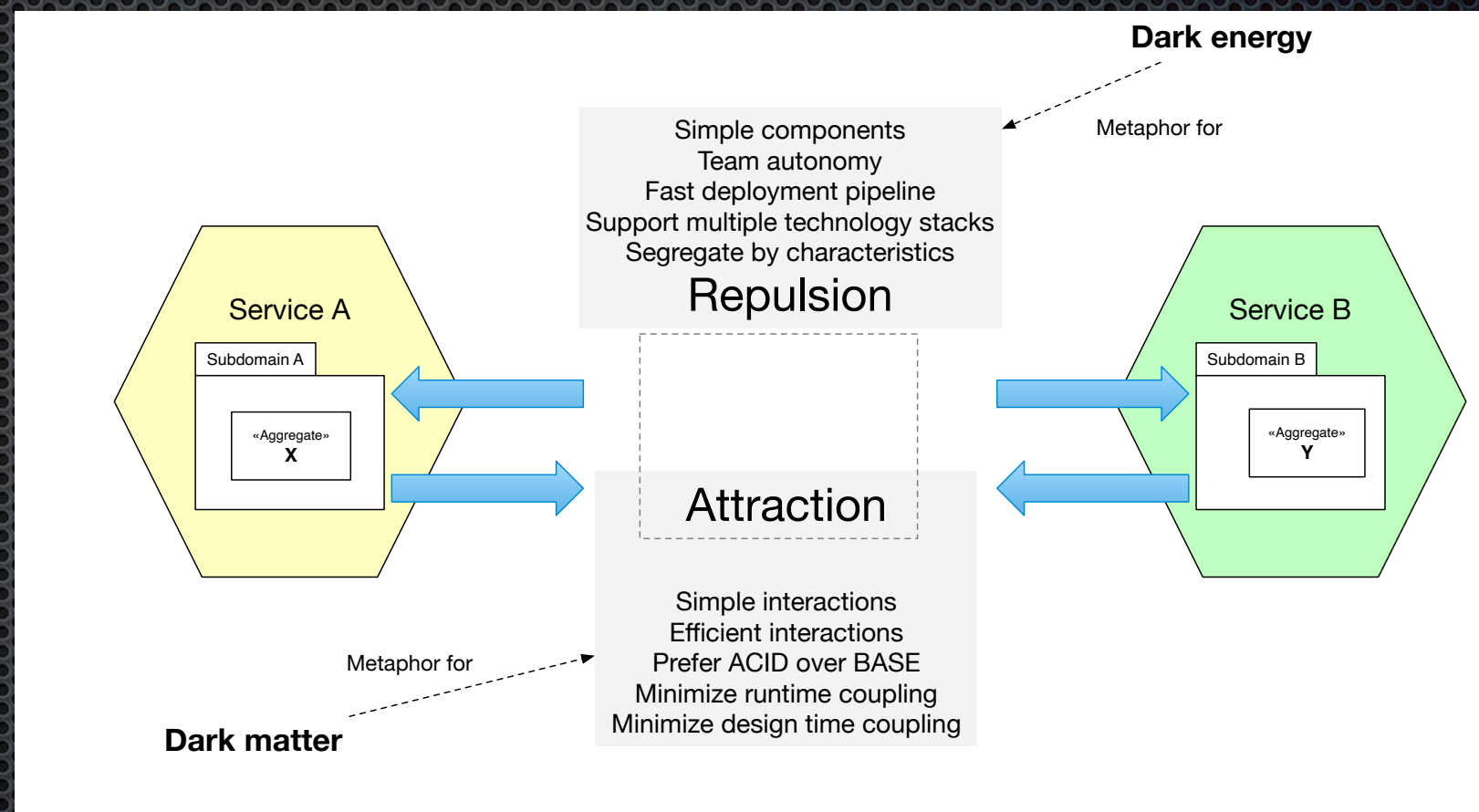
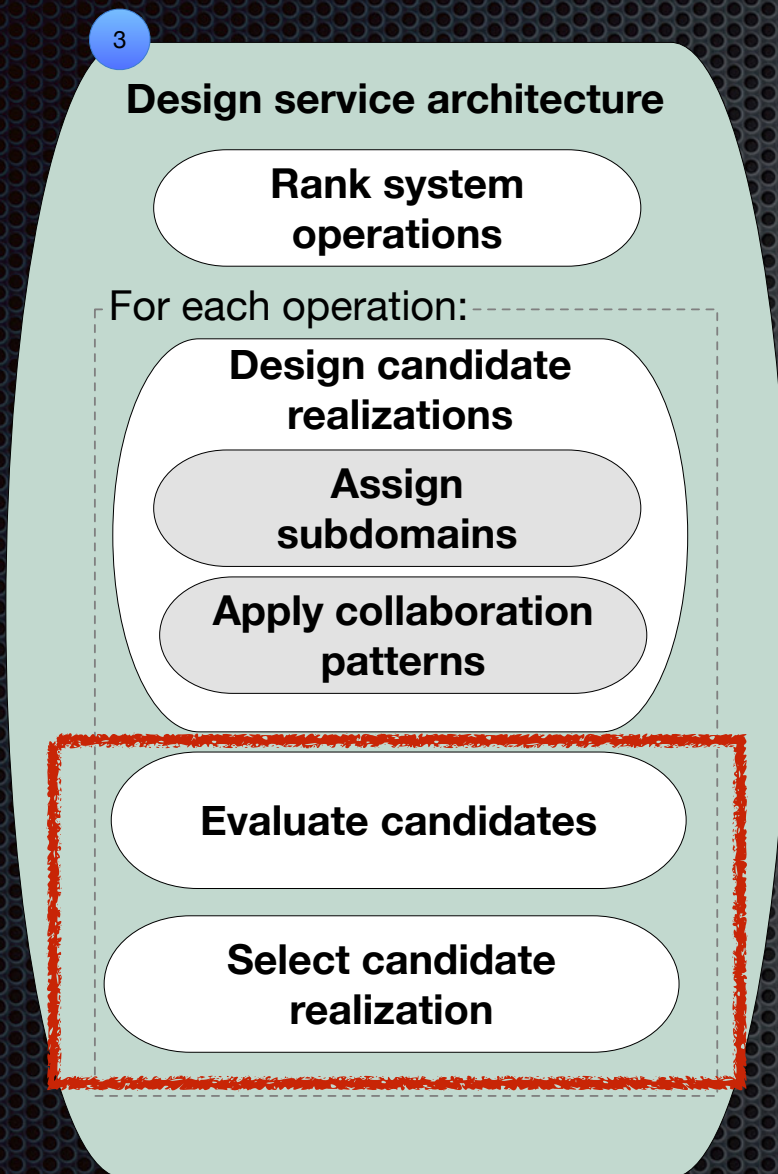


Example candidate realizations for acceptTicket()

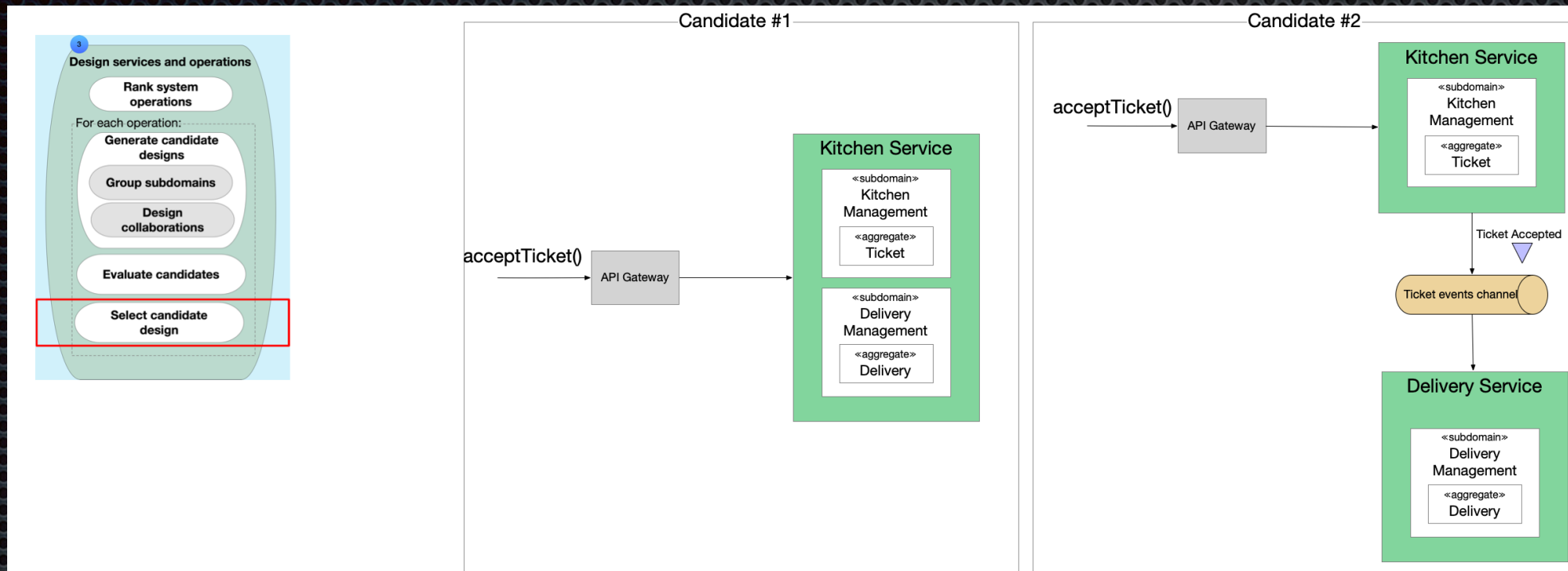


Evaluate candidates using dark energy and dark matter forces and select best one

Forces = criteria for evaluating a solution



Evaluating acceptTicket() realizations

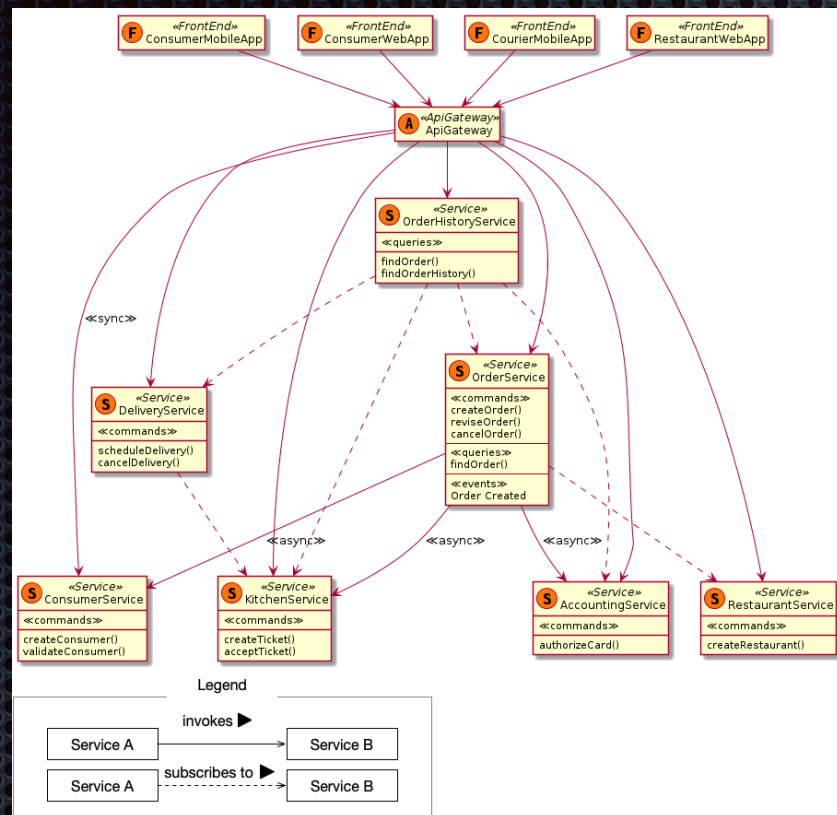


Repulsive dark energy forces			
Simple components		✗	✓
Team autonomy		✗	✓
Fast deployment pipeline		✗	✓
Support multiple technology stacks		✓	✓
Segregate by characteristics		✓	✓
Attractive dark matter forces			
Simple interactions		✓	✓
Efficient interactions		✓	✓
Prefer ACID over BASE		✓	✓
Minimize runtime coupling		✓	✓
Minimize design time coupling		✓	✓

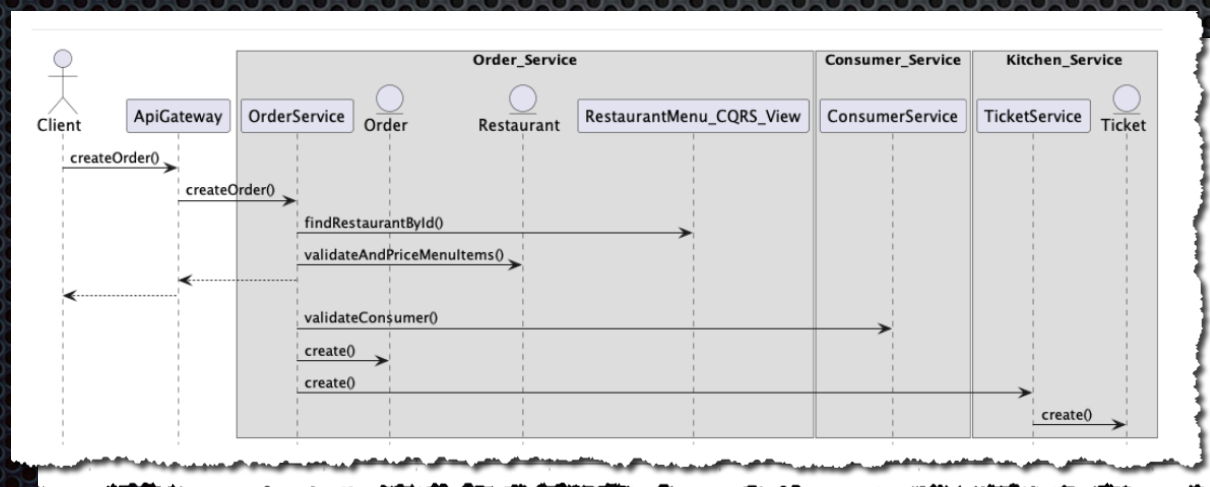
Documenting a microservice architecture

Application-level

Service-level

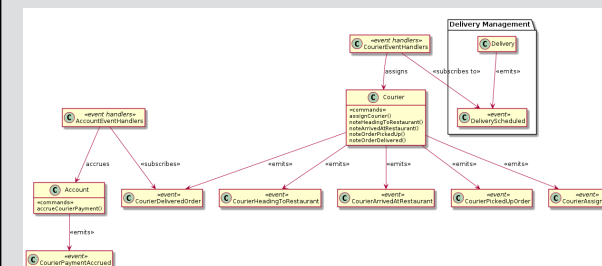


createOrder()



Order Service

Name:	Order Service	
Description:	The Order Service provides an API for creating, revising, and cancelling orders.	
Capabilities		
Order Management		
Service API		
Commands	Queries	Events Published
Synchronous: - createOrder() - reviseOrder() - cancelOrder() Asynchronous: - N/A	getOrder()	Order event channel: - Order Created - Order Authorized - Order Revised - Order Cancelled ...
Non-functional requirements	99.95% availability 1000 orders/second	
Observability		
Key metrics - placed_orders - approved_orders - rejected_orders ...		
Health check endpoint	/actuator/health	
Implementation		
Domain Model Order aggregate		
Dependencies		
Invokes	Subscribes to	
Consumer Service: - validateOrder() Kitchen service: - createTicket() - confirmCreateTicket() - cancelCreateTicket() Accounting Service - authorize()	Restaurant Service - Restaurant Created event - Restaurant Menu Revised event Saga reply channels: - Create Order Saga - Revise Order Saga - Cancel Order Saga	



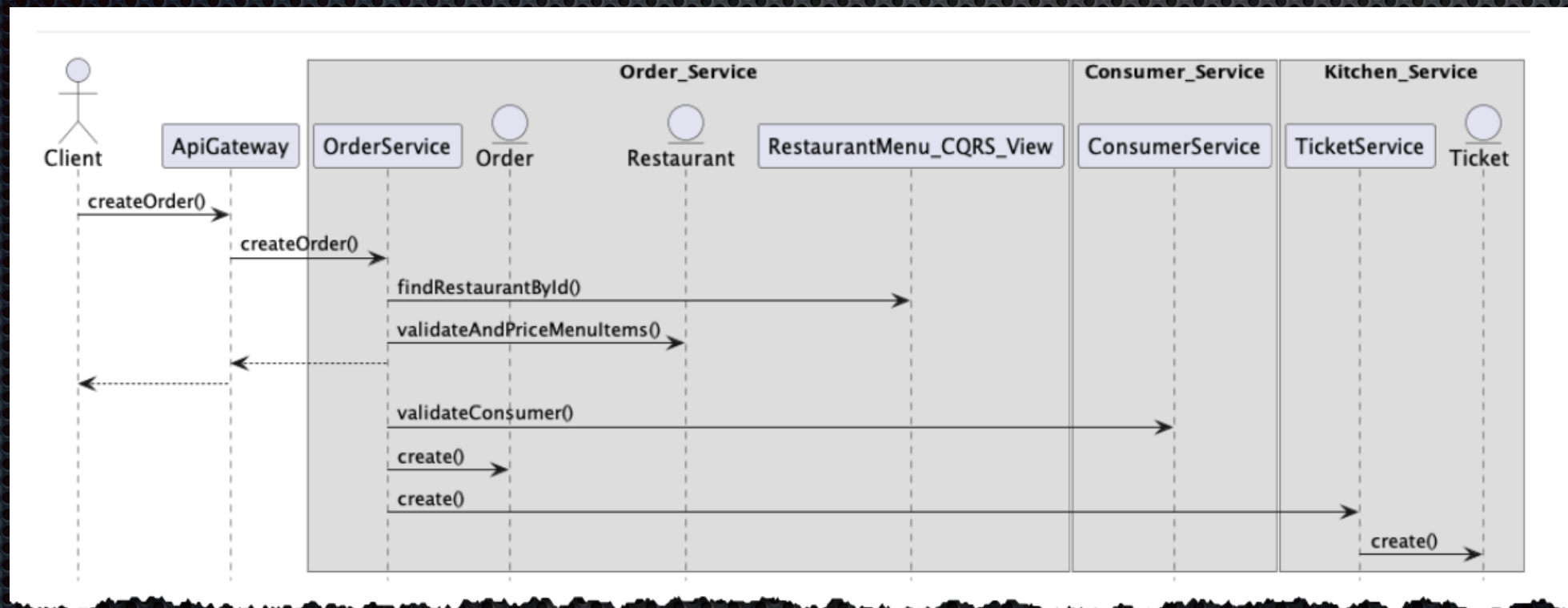
Documenting services: microservice canvas

Order Service		
Name:	Order Service	
Description:		
Capabilities/Requirements		
• Manages consumers • Manages orders		
Service API		
Commands	Queries	Events Published
• createOrder()	• findOrder()	• OrderCreated event
Subdomains		
• Consumer management • Order management		
Non-functional requirements		
Observability		
Key metrics		
Health check endpoint		
Dependencies		
Invokes	Subscribes to	
• Customer Service		

<https://github.com/cer/microservice-canvas>

@crichardson

Documentation: sequence diagrams showing collaboration



Per-operation
/scenario

Documentation: design decisions

Issue:	Realization of <code>acceptTicket()</code>
Decision	Add <code>Delivery Service</code> • Contains <code>Delivery Management</code> subdomain • Has an event handler for <code>TicketAccepted</code> that schedules the delivery Modify <code>Kitchen Service</code> • Entry point for the <code>acceptTicket()</code> operation • The <code>acceptTicket()</code> operation publishes a <code>TicketAccepted</code> event
Alternatives	Alternative: • Add the <code>Delivery Management</code> subdomain to the <code>Kitchen Service</code> • Make the <code>acceptTicket()</code> operation local
Rationale	The chosen design resolves all of the forces despite <code>acceptTicket()</code> being a distributed operation. In contrast, the alternative approach fails to resolve the following forces: • Simple components - the <code>Kitchen Service</code> would have two subdomains, one complicated and the other complex • Team autonomy - the <code>Delivery Management</code> subdomain is complex and should be developed, deployed and operated independently • Fast deployment pipeline - the <code>Kitchen Service</code> would have to be deployed whenever the <code>Delivery Management</code> subdomain changes

Agenda

- ✦ Introduction to Assemblage
- ✦ Step 1: discover system operations
- ✦ Step 2: design subdomains
- ✦ Step 3: define the service architecture
- ✦ Step 4: evaluate the architecture
- ✦ Step 5: refactor the architecture

Step #4: evaluate the architecture

Evaluation = does it?

A microservice architecture must

Service

Subdomain



Minimize repulsion:

- Simple components
- Team autonomy
- Fast deployment pipeline
- Support multiple technology stacks
- Segregate by characteristics



Subdomain

Minimize attraction:

- Simple interactions
- Efficient interactions
- Prefer ACID over BASE
- Minimize runtime coupling
- Minimize design time coupling

Service

Subdomain

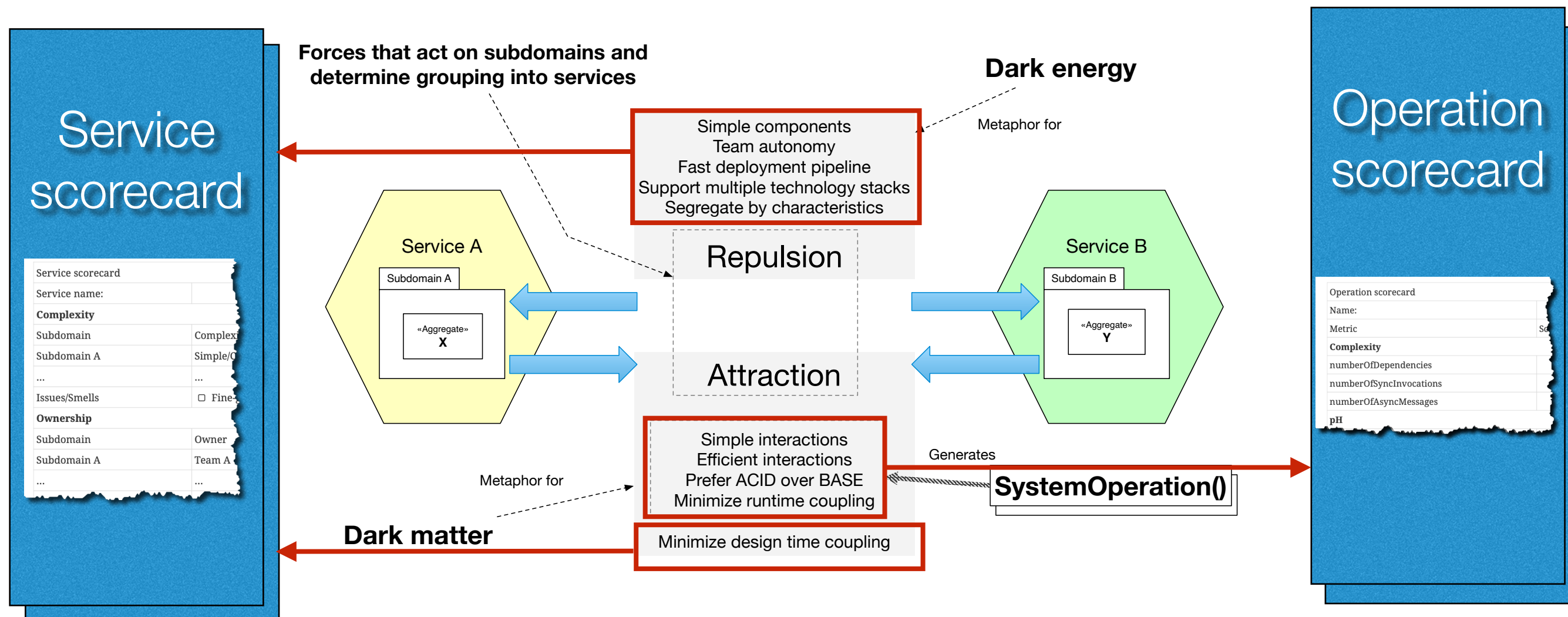


Minimize repulsion

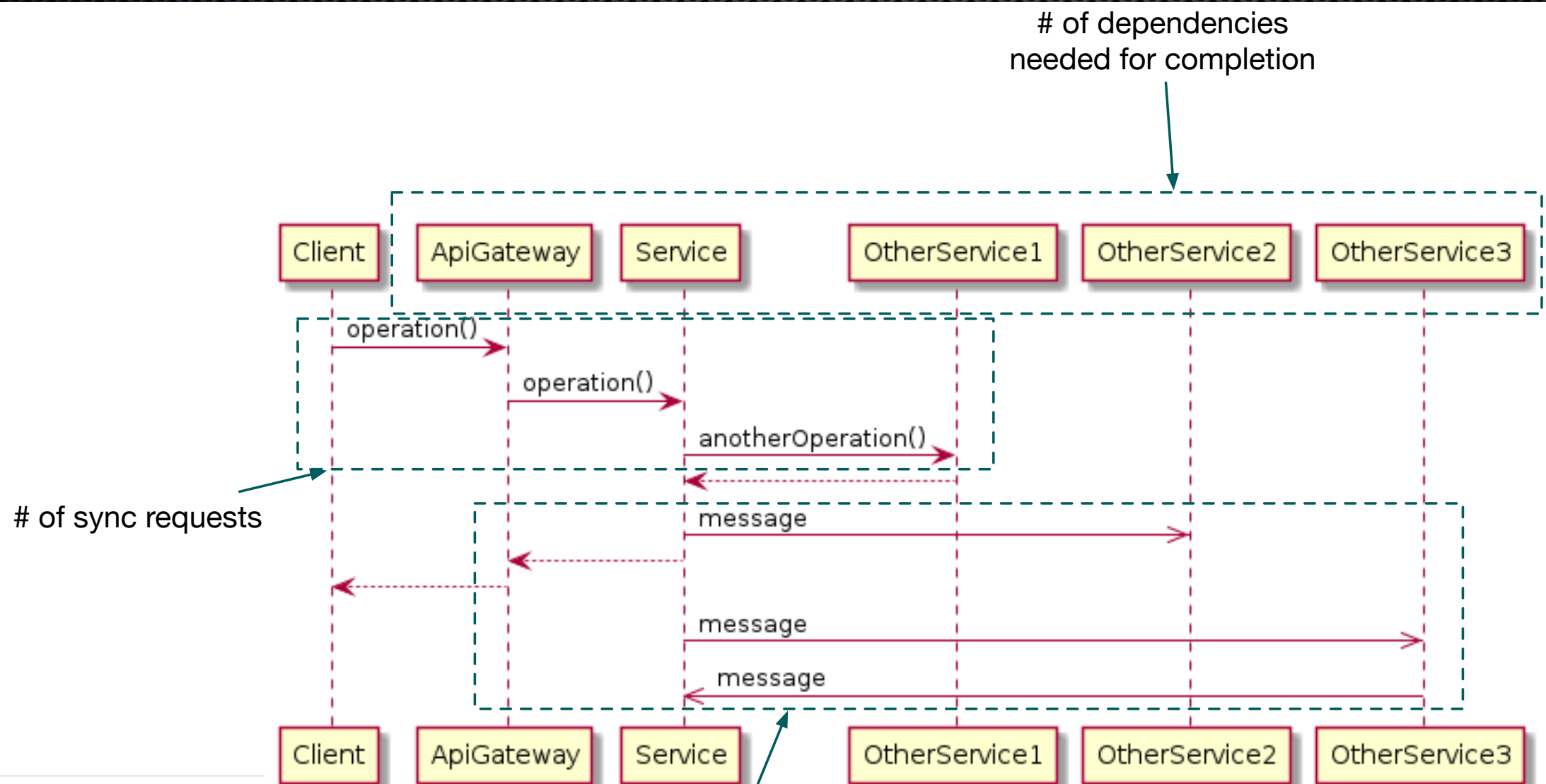


Subdomain

Evaluation => service and operation scorecards



Evaluating interaction complexity



Operation scorecard

Name:

Metric

Score

of async messages

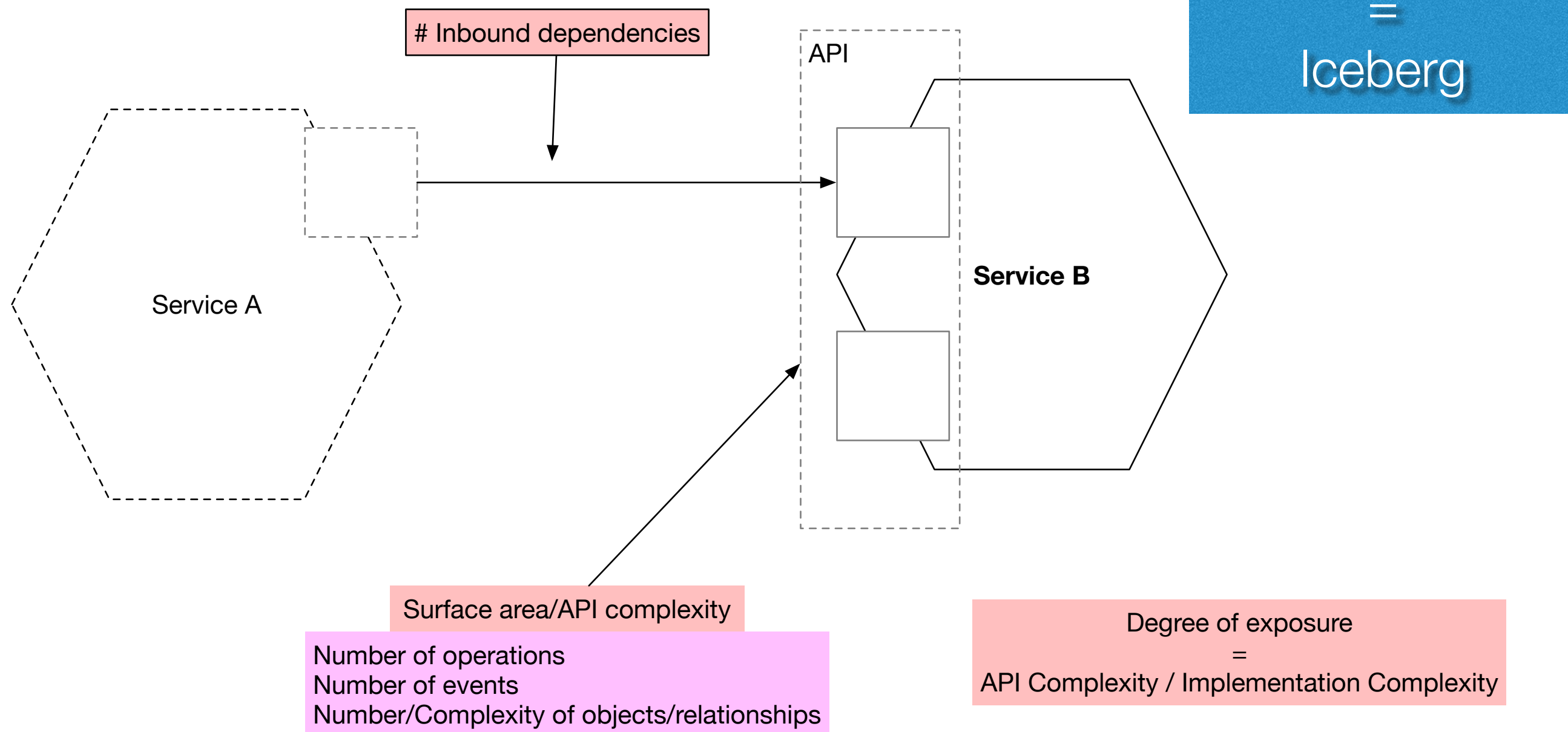
Complexity

numberOfDependencies

numberOfSyncInvocations

numberOfAsyncMessages

Evaluating design-time coupling - API



Agenda

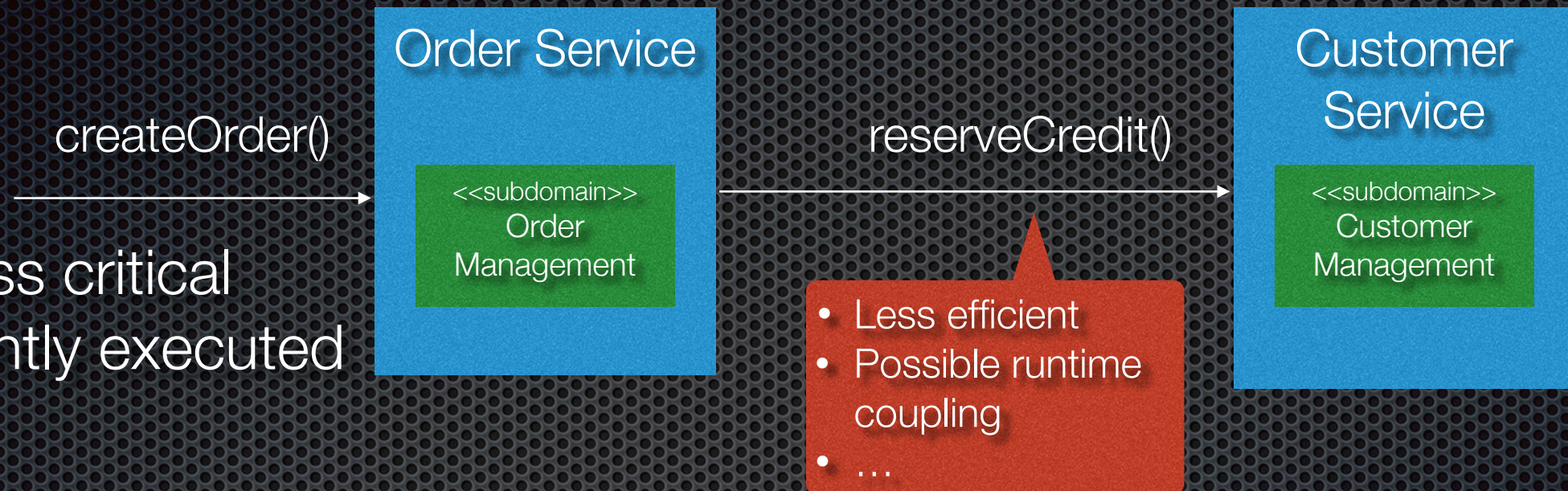
- ✦ Introduction to Assemblage
- ✦ Step 1: discover system operations
- ✦ Step 2: design subdomains
- ✦ Step 3: define the service architecture
- ✦ Step 4: evaluate the architecture
- ✦ Step 5: refactor the architecture

Step #5: refactor to improve the architecture

- ✦ Redesign system operations
- ✦ Redefine services by moving subdomains
- ✦ Redefine subdomains to facilitate moving between services
 - ✦ Move aggregates between subdomains
 - ✦ Split aggregates (and then move)
- ✦ Revise requirements, e.g. allow eventual consistency

Example: improve createOrder()

- Business critical
- Frequently executed



Move *Credit Management* to Order Service



 [@crichardson](https://twitter.com/crichardson) chris@chrisrichardson.net

Questions?

adopt.microservices.io